

Sesión 3

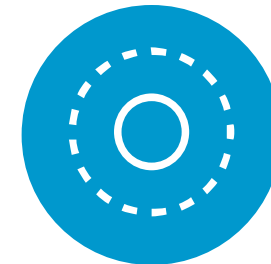
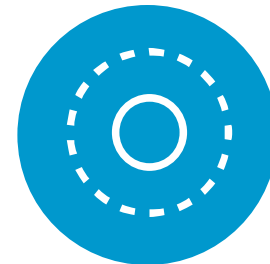
11 Mayo 2025

- 1. Material de Apoyo**
- 2. Apache Kafka**
- 3. BBDD NoSQL**
- 4. Redshift**

1/ Material de Apoyo

UF1 Aplicación de técnicas de integración, procesamiento y análisis de información.

L02. Técnicas y procesos de extracción de la información de los datos .

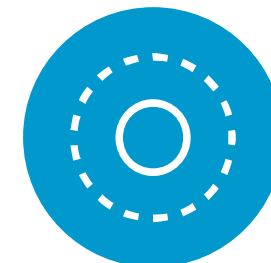
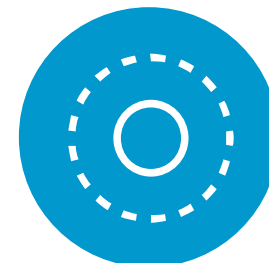


UF2 Técnicas de representación y minería de datos

L03. Métodos y algoritmos de minería de datos

UF 3 Sistemas de almacenamiento

Sistemas de gestión almacenamiento HDFS.
Apache Kafka, Motores y bbdd NoSQL



UF 4 Conceptos y visualizacion

L04 Herramientas de visualización de datos

- Página de Inicio
- Temas
- Programación semanal
- Archivos
- Clases en directo
- Anuncios
- Foros de discusión
- Tareas
- Calificaciones
- Calificaciones finales
- Personas
- Evaluaciones
- Módulos
- Páginas
- Programa del curso
- Competencias
- Colaboraciones

Programa Profesional en Inteligencia Artificial y Data Science - Octubre 2024 (PER 11324) > ... > Material de Apoyo

Buscar archivos

0 items seleccionados

+ Carpeta

Cargar

Programa Profesional en Inteligencia Artificial y Data Science

Actividades

Multimedia cargada

Presentaciones clases

Módulo 1: Capacidades de Inteligencia Artificial

Módulo 2: Lenguajes y Desarrollo

Módulo 3: Aprendizaje Automático

Módulo 4: Analítica Escalable

Material de Apoyo

Actividades de Evaluación

Actividad 7

Actividad 8

Docker Contenedores

Nombre	Fecha de creación	Fecha de modificación	Modificado por	Tamaño
Actividades de Evaluación	Martes			
Docker Contenedores	23 de abr de 2025			
Labs de AWS	27 de abr de 2025			
Practica HDFS	23 de abr de 2025			
Practicas Kafka	23 de abr de 2025			

96% de 524 MB utilizados

Todos mis archivos

1/ BBDD NoSQL

Introducción a las bases de datos NoSQL

¿Por qué BBBD NoSQL?

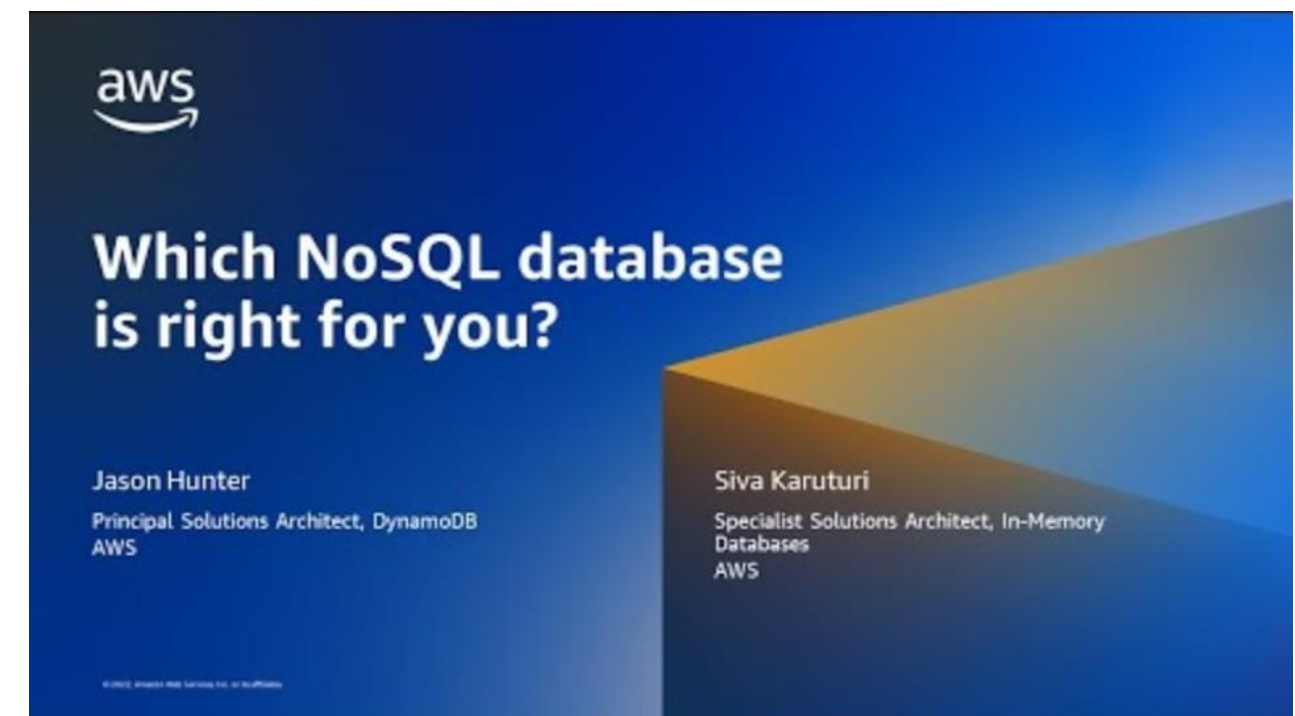
Volumen Masivo: Cada día generamos cantidades de datos sin precedentes. Desde transacciones en línea hasta datos de sensores y redes sociales, el volumen crece a un ritmo vertiginoso. ¡Estamos hablando de petabytes y exabytes de información!

Velocidad Vertiginosa: Los datos fluyen a una velocidad asombrosa. Piénsalo: tweets por segundo, lecturas de sensores en tiempo real, logs de servidores... Necesitamos procesar y analizar esta información en tiempo real o casi real.

Variedad Desafiante: Los datos ya no vienen solo en filas y columnas ordenadas. Ahora tenemos texto, imágenes, videos, audio, datos de geolocalización, datos de sensores... ¡una heterogeneidad que las bases de datos tradicionales luchan por manejar!

Veracidad en Cuestión: Con tanta información, asegurar la calidad y la fiabilidad de los datos se vuelve crucial. ¿Podemos confiar en lo que estamos analizando?

Valor Oculto: Enormes cantidades de datos sin procesar tienen poco valor. El verdadero desafío del Big Data es extraer información significativa, patrones y conocimiento que impulsen decisiones y generen valor para las organizaciones..



El Cuello de Botella de las Bases de Datos Relacionales

- Las bases de datos relacionales tradicionales, con su estructura rígida y su enfoque en la escalabilidad vertical, a menudo se encuentran con **limitaciones significativas** al enfrentarse a este panorama del Big Data.
- **Escalabilidad Compleja y Costosa:** Aumentar la capacidad de un único servidor (escalabilidad vertical) puede volverse prohibitivamente caro y alcanzar límites físicos.
- **Dificultad para Manejar Datos No Estructurados:** Su diseño basado en esquemas fijos no se adapta bien a la flexibilidad y la diversidad de los datos modernos.
- **Rendimiento Degradado con Grandes Volúmenes:** Las consultas complejas en tablas masivas pueden volverse lentas e ineficientes.



**Limitaciones
de las
Bases de Datos
Relacionales**

Las bases de datos NoSQL surgieron como una solución para manejar las necesidades de las aplicaciones modernas, que se enfrentan a un gran volumen, variedad y velocidad de datos (las 3 V's del Big Data). Algunas de las necesidades clave que resuelven son:

- **Escalabilidad horizontal:** Las bases de datos NoSQL permiten distribuir los datos en múltiples nodos de forma eficiente, algo que las bases de datos relacionales no hacen tan bien. Esto es fundamental en aplicaciones como redes sociales, donde la cantidad de usuarios y datos crece exponencialmente.
- **Flexibilidad de almacenamiento:** Muchas aplicaciones actuales manejan datos no estructurados o semi-estructurados, como JSON, XML o archivos multimedia. Las bases de datos NoSQL permiten almacenar estos datos sin la necesidad de un esquema fijo, lo que reduce la complejidad.
- **Rendimiento en tiempo real:** Las aplicaciones como los sistemas de recomendación, el análisis en tiempo real o la publicidad digital requieren tiempos de respuesta muy rápidos con grandes volúmenes de datos. Las bases de datos NoSQL están optimizadas para estas operaciones rápidas de lectura y escritura.

Ventajas de las bases de datos NoSQL

Flexibilidad

Las bases de datos NoSQL pueden almacenar y administrar datos en varios formatos, incluidos JSON, XML, pares clave-valor y más. Esta flexibilidad permite a las organizaciones adaptarse a los requisitos de datos cambiantes sin alterar el esquema de su base de datos.

Escalabilidad

Las bases de datos NoSQL están hechas para crecer de forma horizontal. Pueden trabajar con mucha información y mucha gente repartiendo los datos entre varios puntos o grupos, lo que asegura que la velocidad no se pierda a medida que hay más demanda.

Alta disponibilidad

Tienen sistemas incorporados para asegurar que no se caigan, ni siquiera cuando hay problemas con los equipos o la conexión. Esto es muy importante para aplicaciones que no pueden parar, que necesitan estar funcionando siempre.

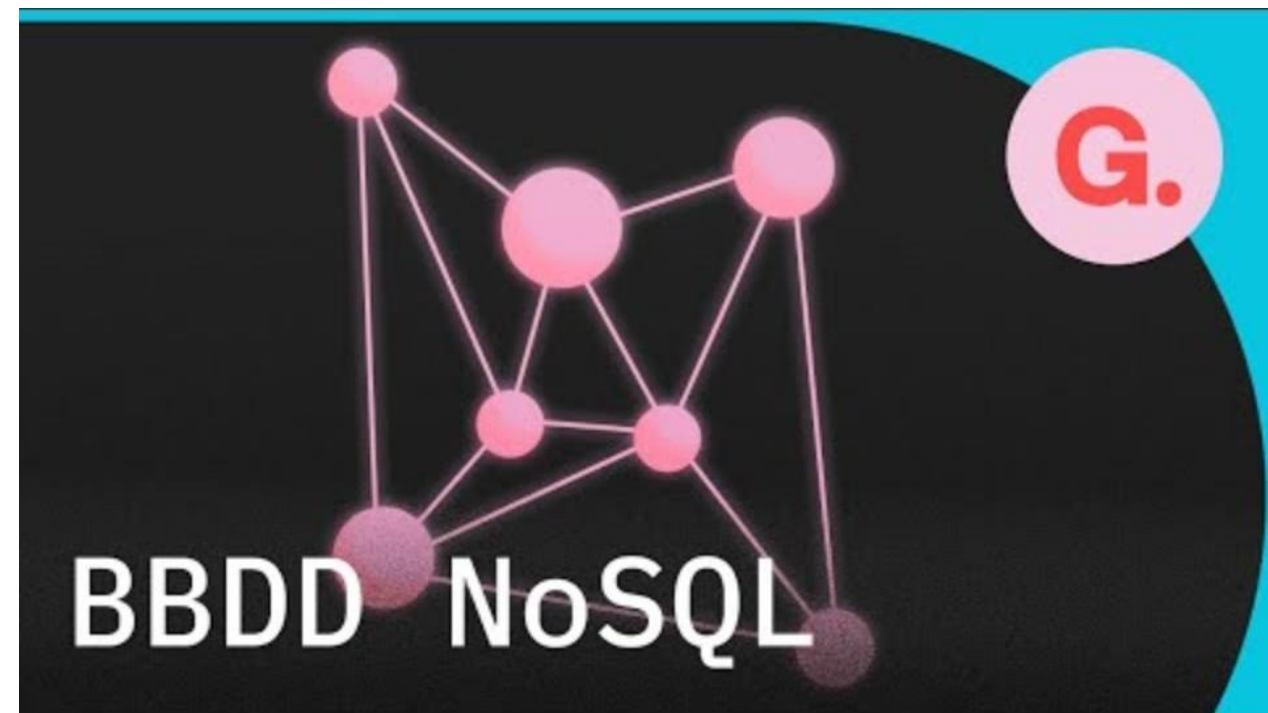
Funcionalidad mejorada

Las bases de datos NoSQL tienen tipos de datos que se adaptan a cada situación, que nos ofrecen modelos de datos distintos y formas de trabajar con ellos muy sencillas.

La Respuesta: Un Nuevo Enfoque con las Bases de Datos NoSQL

Para superar estos desafíos, surge un conjunto de tecnologías innovadoras: las bases de datos NoSQL.

Estas bases de datos ofrecen nuevas formas de organizar y gestionar la información, diseñadas específicamente para la escalabilidad, la flexibilidad y el rendimiento en el mundo del Big Data.



El término **NoSQL** significa "Not Only SQL" o "No SQL", refiriéndose a un conjunto de sistemas de gestión de bases de datos que no siguen el modelo tradicional de bases de datos relacionales basadas en SQL. Surgieron como respuesta a las limitaciones de escalabilidad y flexibilidad de las bases de datos relacionales, especialmente cuando las empresas empezaron a enfrentarse a volúmenes masivos de datos, como en el caso de redes sociales, comercio electrónico, y análisis en tiempo real.

Características principales de NoSQL:

- **Esquemas flexibles:** A diferencia de las bases de datos SQL, las bases de datos NoSQL no requieren un esquema fijo. Los datos pueden tener diferentes estructuras dentro de la misma colección o tabla.
- **Distribución horizontal:** Permiten la partición de datos (sharding) en múltiples servidores para mejorar el rendimiento y la escalabilidad.
- **Alta disponibilidad y tolerancia a fallos:** Están diseñadas para entornos distribuidos, lo que las hace más resistentes a fallos de hardware o red.



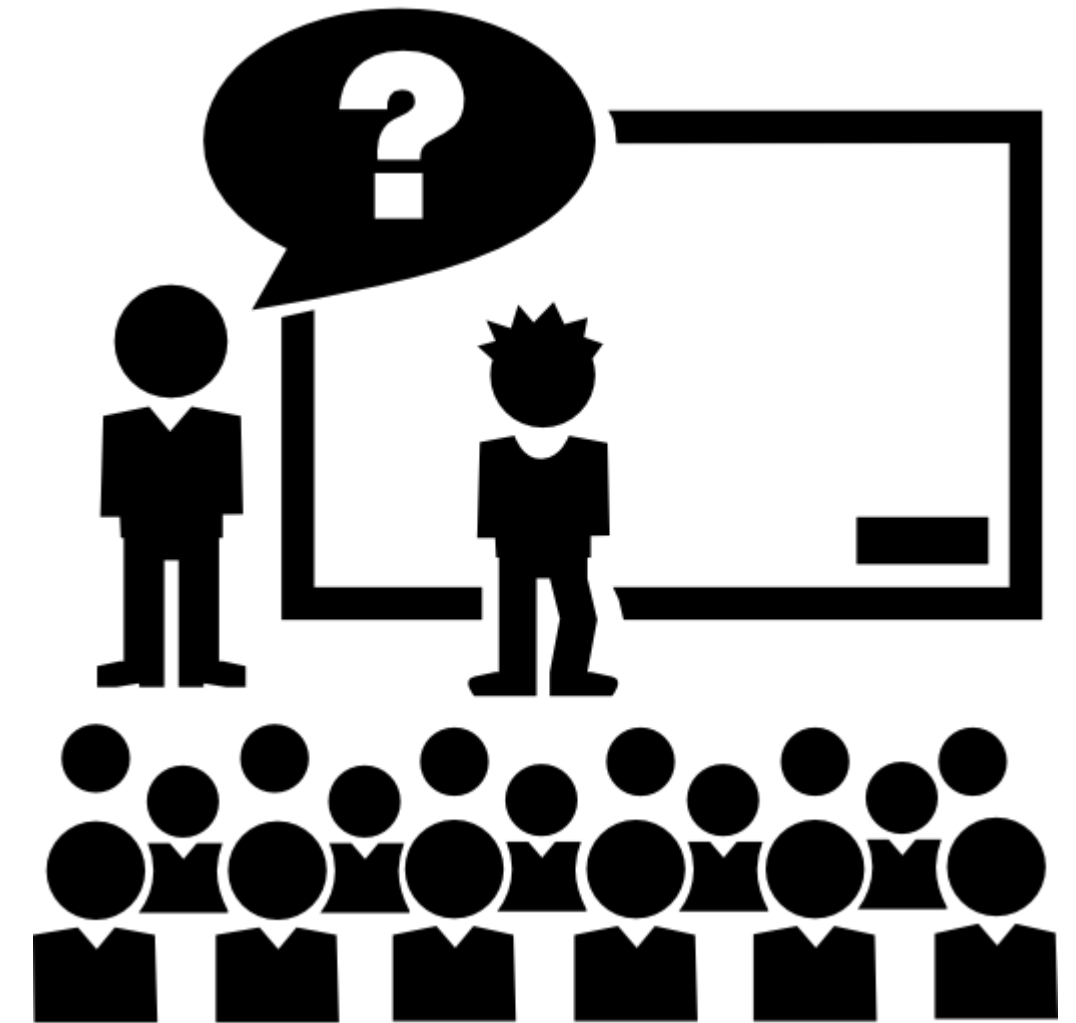
SQL (Structured Query Language) ha sido la base de los sistemas de gestión de bases de datos relacionales (RDBMS) durante décadas, proporcionando un modelo estructurado y estandarizado para manejar datos.

- **Estructura de datos:**
 - **SQL:** Basadas en tablas con filas y columnas, lo que requiere que todos los datos sigan un esquema predefinido.
 - **NoSQL:** Pueden almacenar datos no estructurados o semi-estructurados (JSON, documentos, pares clave-valor), lo que las hace más flexibles.
- **Escalabilidad:**
 - **SQL:** Escala verticalmente (añadiendo más recursos a un solo servidor).
 - **NoSQL:** Escala horizontalmente (añadiendo más servidores), lo que es ideal para aplicaciones con un gran volumen de datos.
- **Consistencia vs Disponibilidad:** Introducir el **Teorema CAP** (Consistencia, Disponibilidad y Tolerancia a particiones).
 - **SQL:** En su mayoría, priorizan la **consistencia fuerte** (todos los usuarios ven la misma versión de los datos en cualquier momento).
 - **NoSQL:** A menudo optan por **disponibilidad y tolerancia a particiones** en sistemas distribuidos, aceptando cierto grado de inconsistencia temporal (consistencia eventual).

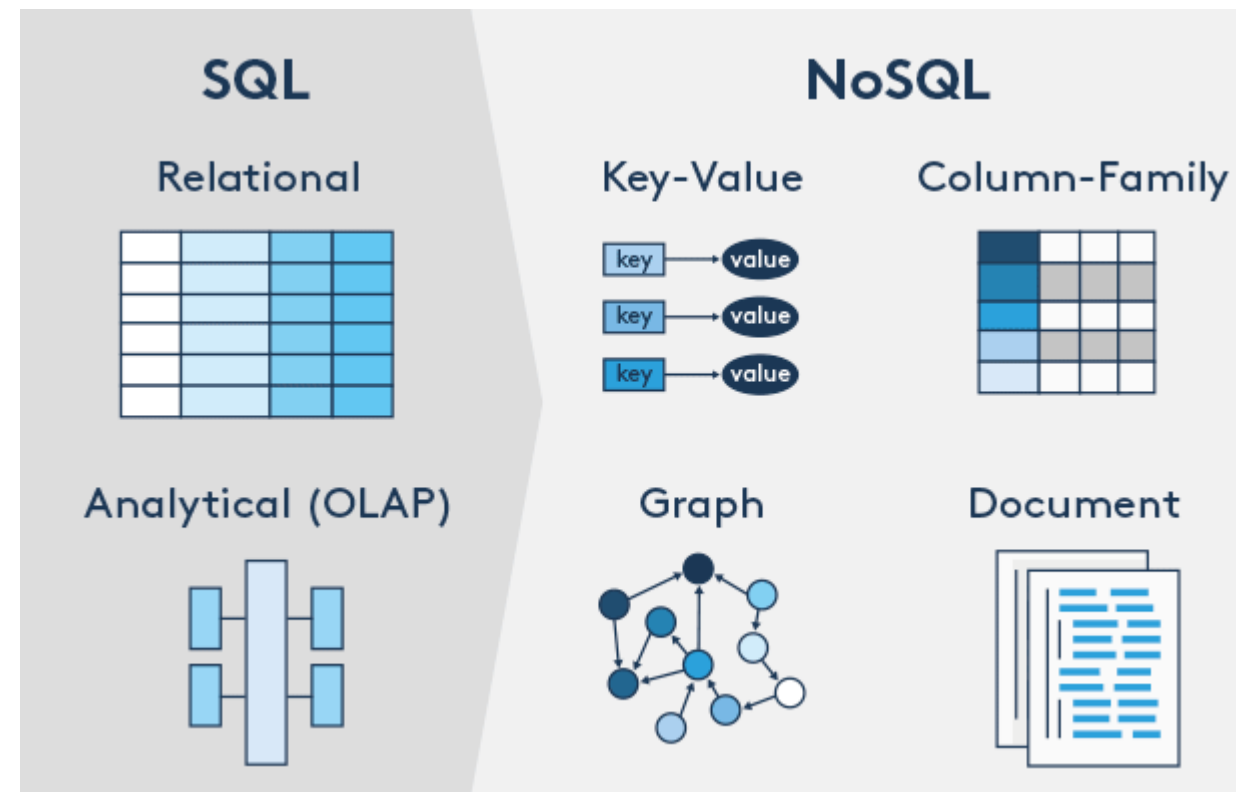
Característica	SQL	NoSQL
Estructura de datos	Esquema fijo (tablas)	Esquema flexible
Consistencia	Consistencia fuerte	Consistencia eventual
Escalabilidad	Vertical	Horizontal
Tipo de transacciones	ACID (transacciones completas)	BASE (transacciones más ligeras)
Aplicaciones típicas	ERP, CRM, contabilidad	Redes sociales, Big Data

Pregunta a la clase: "¿Cuáles de las aplicaciones que utilizan diariamente creen que usan NoSQL?"

(Ejemplos: redes sociales como Meta o X, aplicaciones de mensajería como WhatsApp, plataformas de streaming como Netflix, AEAT)".



En el ecosistema de bases de datos NoSQL, existen cuatro tipos principales, cada uno de ellos diseñado para casos de uso específicos según la estructura y las necesidades del tipo de datos que manejan. Estos son: bases de datos de **clave-valor**, **documentos**, **columnar** y **grafos**.



Comparación con las bases de datos SQL tradicionales

Esquema Fijo vs. Flexible

Las bases de datos SQL utilizan un esquema fijo, mientras que las bases de datos NoSQL ofrecen esquemas flexibles que se adaptan a diversos formatos.

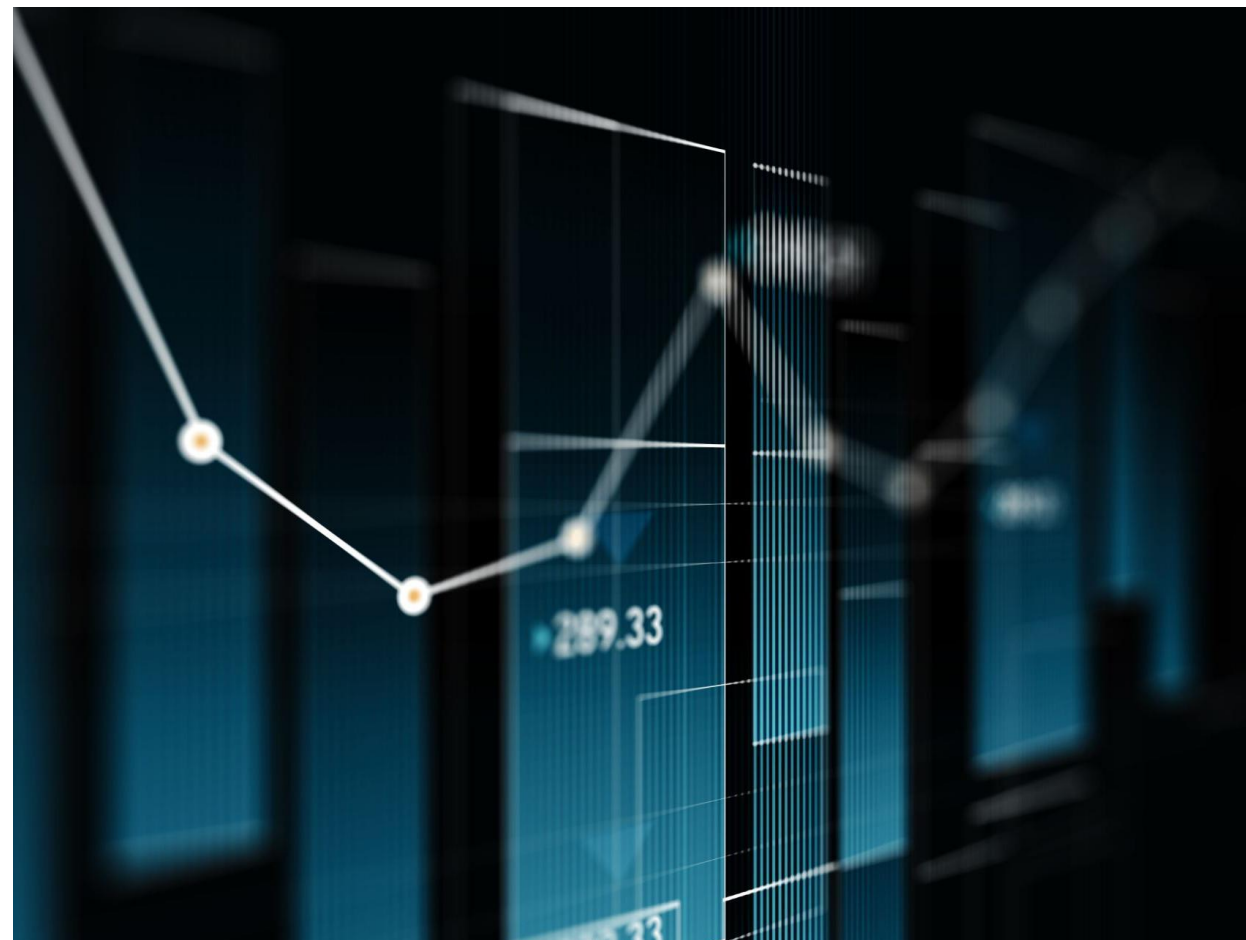
Lenguaje de Consulta

SQL utiliza un lenguaje de consulta estructurado, mientras que NoSQL permite más libertad en la forma de acceder y manipular los datos.

Adaptabilidad a Cambios

Las bases de datos NoSQL son altamente adaptables a los cambios en los requisitos de las aplicaciones, a diferencia de las bases de datos SQL más rígidas.

Ventajas y desventajas de NoSQL



Escalabilidad Horizontal

NoSQL permite escalabilidad horizontal, lo cual es crucial para manejar grandes volúmenes de datos de manera eficiente.

Flexibilidad de Datos

Ofrecen flexibilidad en el almacenamiento de datos no estructurados, facilitando la adaptación a diferentes tipos de información.

Rendimiento Mejorado

NoSQL puede proporcionar un mejor rendimiento en comparación con las bases de datos relacionales, especialmente en cargas de trabajo específicas.

Desafíos de Consistencia

La menor consistencia en algunos modelos NoSQL puede ser un reto importante para ciertas aplicaciones que requieren alta integridad de datos.

2 / Categoría BBDD NoSQL

Bases de datos de documentos

Almacenamiento flexible de datos

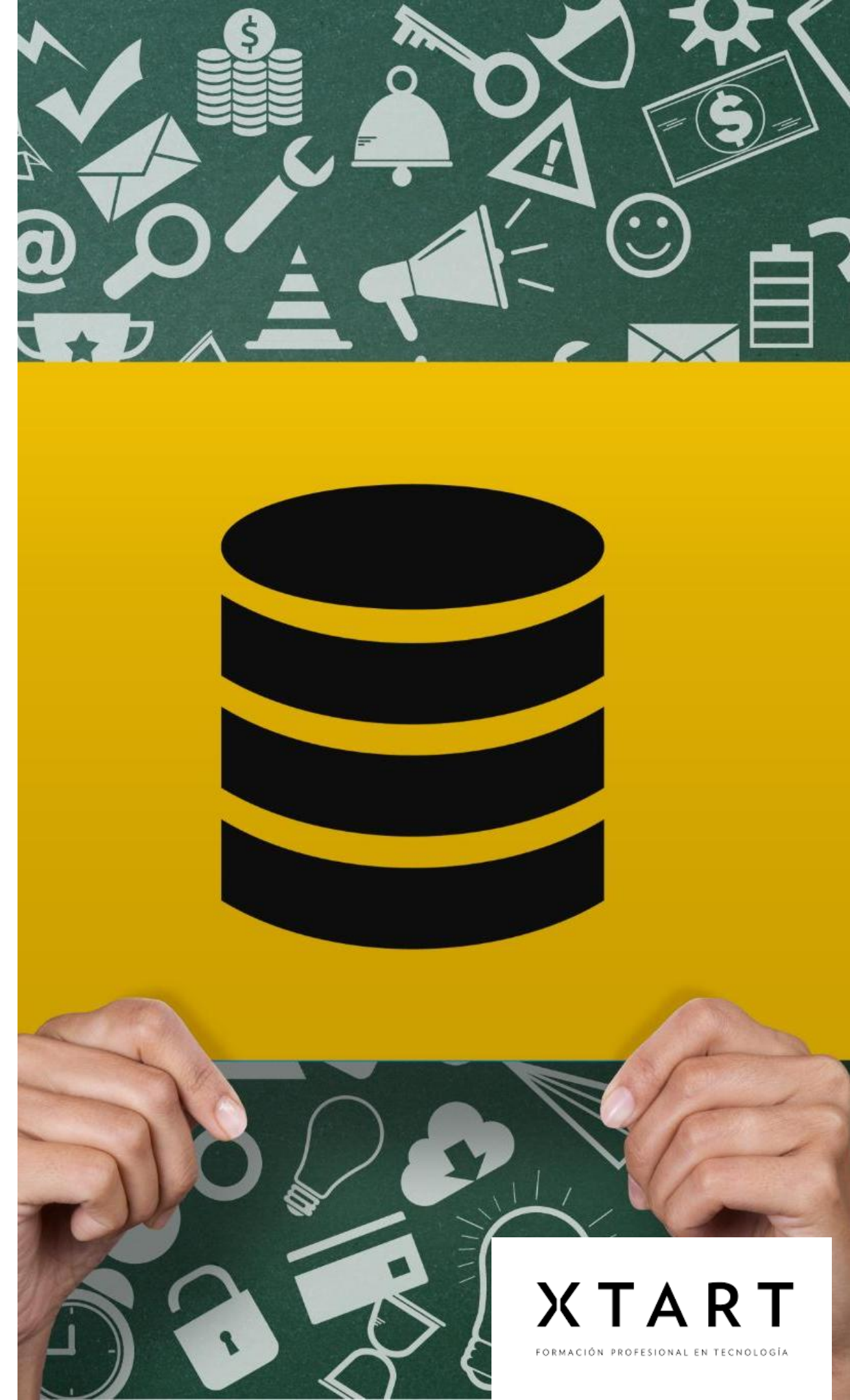
Las bases de datos de documentos permiten almacenar datos en documentos con estructuras variables, proporcionando gran flexibilidad en el modelo de datos.

Acceso rápido a datos complejos

Son ideales para aplicaciones que requieren un acceso rápido y fácil a datos complejos, mejorando la eficiencia en el manejo de información.

Uso de JSON y BSON

Las bases de datos de documentos utilizan formatos como JSON y BSON para almacenar y intercambiar datos, facilitando su manipulación.



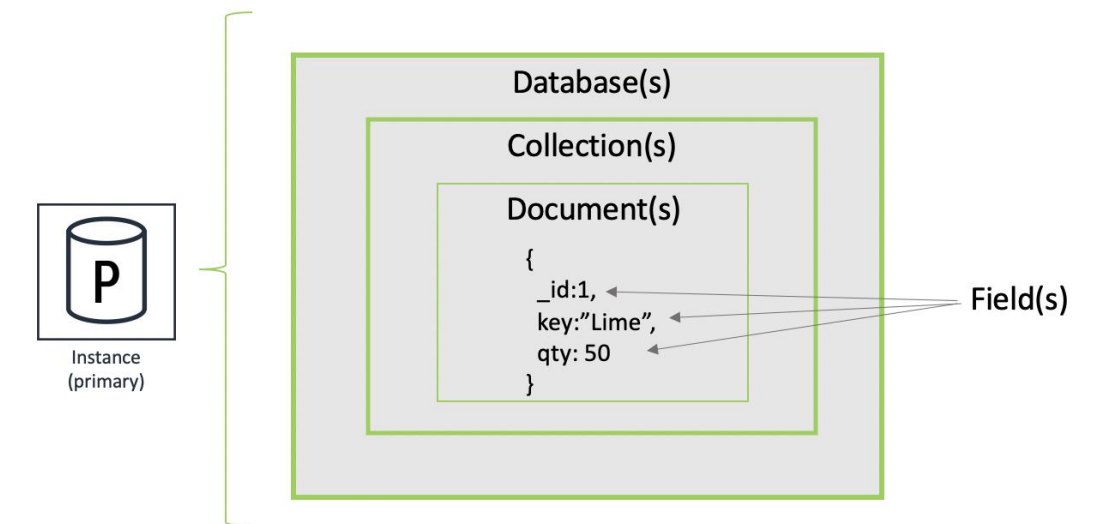
Bases de datos de documentos

Descripción:

- Almacena datos en "documentos", que son estructuras de datos semiestructuradas (generalmente JSON, BSON o XML).
- Los documentos pueden tener diferentes campos y tipos de datos, incluso dentro de la misma colección.
- Organiza los documentos en "colecciones" (análogas a las tablas en las bases de datos relacionales)

Características Clave:

- **Esquema flexible:** Cada documento puede tener su propia estructura.
- **Ricos tipos de datos:** Soporte para arrays, objetos anidados y otros tipos de datos complejos.
- **Consultas expresivas:** Permite realizar consultas complejas dentro de los documentos.
- **Indexación:** Soporta la creación de índices para optimizar las consultas.
- **Casos de Uso Típicos en Big Data:**
 - **Catálogos de productos:** Almacenar información detallada sobre productos con diferentes atributos.
 - **Gestión de contenido:** Almacenar artículos, publicaciones de blog y otros contenidos con campos variados.
 - **Perfiles de usuario complejos:** Almacenar información detallada sobre usuarios con diferentes campos y preferencias.
 - **Datos de sensores:** Almacenar lecturas de sensores con diferentes mediciones.
 - **Datos de eventos:** Registrar eventos con diferentes propiedades.
 - **Plataformas de comercio electrónico:** Carritos de compras, historial de pedidos.



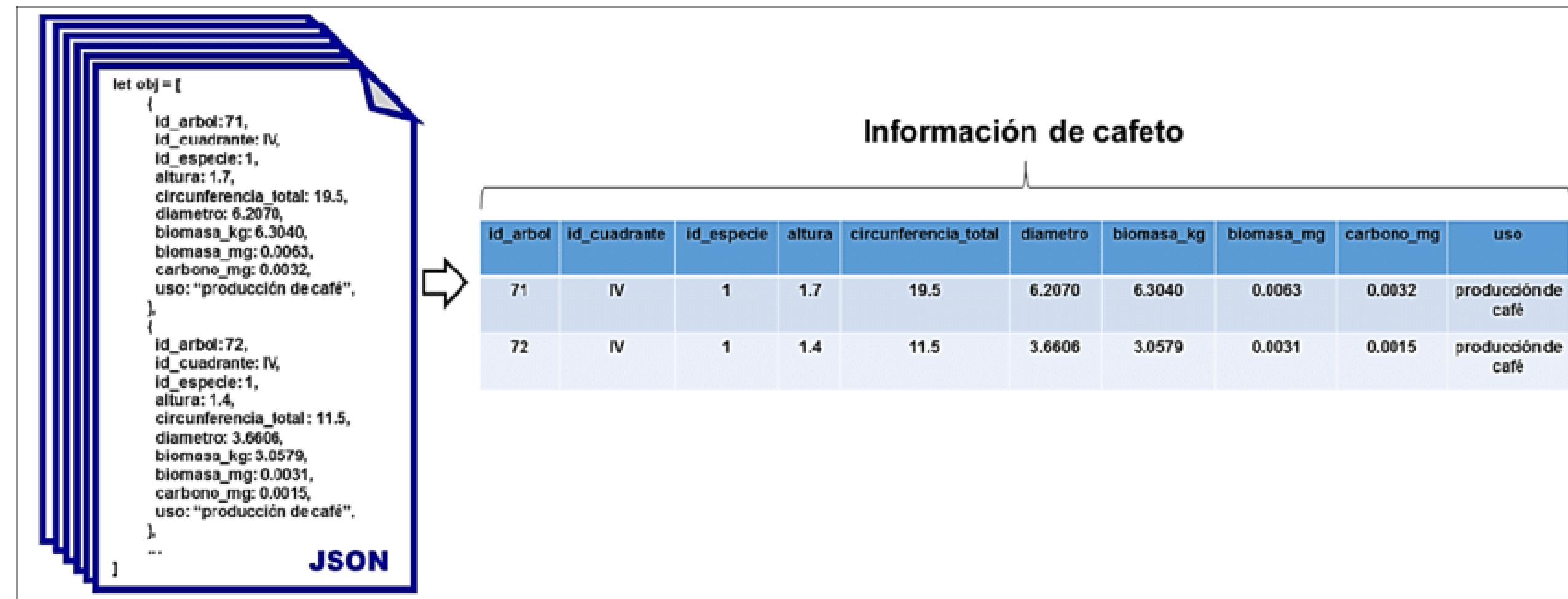
Bases de datos de documentos

Ventajas:

- Esquema flexible que permite una gran agilidad de desarrollo.
- Maneja bien los datos semiestructurados.
- Las consultas pueden ser muy expresivas.
- Fácil de escalar horizontalmente.

Desventajas:

- La consistencia eventual es común en sistemas distribuidos.
- Las consultas complejas pueden ser ineficientes si no se indexan adecuadamente.
- Uniones (joins) complejas pueden ser difíciles de realizar.



Las bases de datos de documentos almacenan los datos en formato de documentos, habitualmente en **JSON**, **BSON** o **XML**. Cada documento es una entidad autónoma que puede tener un esquema flexible, lo que significa que los diferentes documentos en una misma colección no necesitan tener el mismo formato.

- **Funcionamiento:** Los datos se organizan en documentos, cada uno de los cuales puede tener diferentes campos. Los documentos se agrupan en colecciones y se pueden indexar para facilitar consultas rápidas.

Casos de uso:

- **Aplicaciones web:** Que manejan grandes cantidades de datos semiestructurados, como perfiles de usuario, registros de actividad, o información de productos.
- **Sistemas de gestión de contenido (CMS):** Donde los datos tienen estructura flexible o cambiante.

Ejemplos:

- **MongoDB:** Es el sistema de base de datos de documentos más popular, ideal para aplicaciones que requieren alta flexibilidad en el manejo de datos.
- **CouchDB:** Utiliza JSON para almacenar datos y ofrece una API HTTP para realizar consultas.

Visualización (ejemplo de MongoDB):

```
json Copiar código
{
  "_id": "603c9c2b2f1f4f07a44f05a9",
  "name": "John Doe",
  "email": "johndoe@example.com",
  "orders": [
    { "order_id": "001", "total": 100, "status": "shipped" },
    { "order_id": "002", "total": 200, "status": "pending" }
  ]
}
```


Bases de datos de claves-valor

Modelo de Almacenamiento Simple

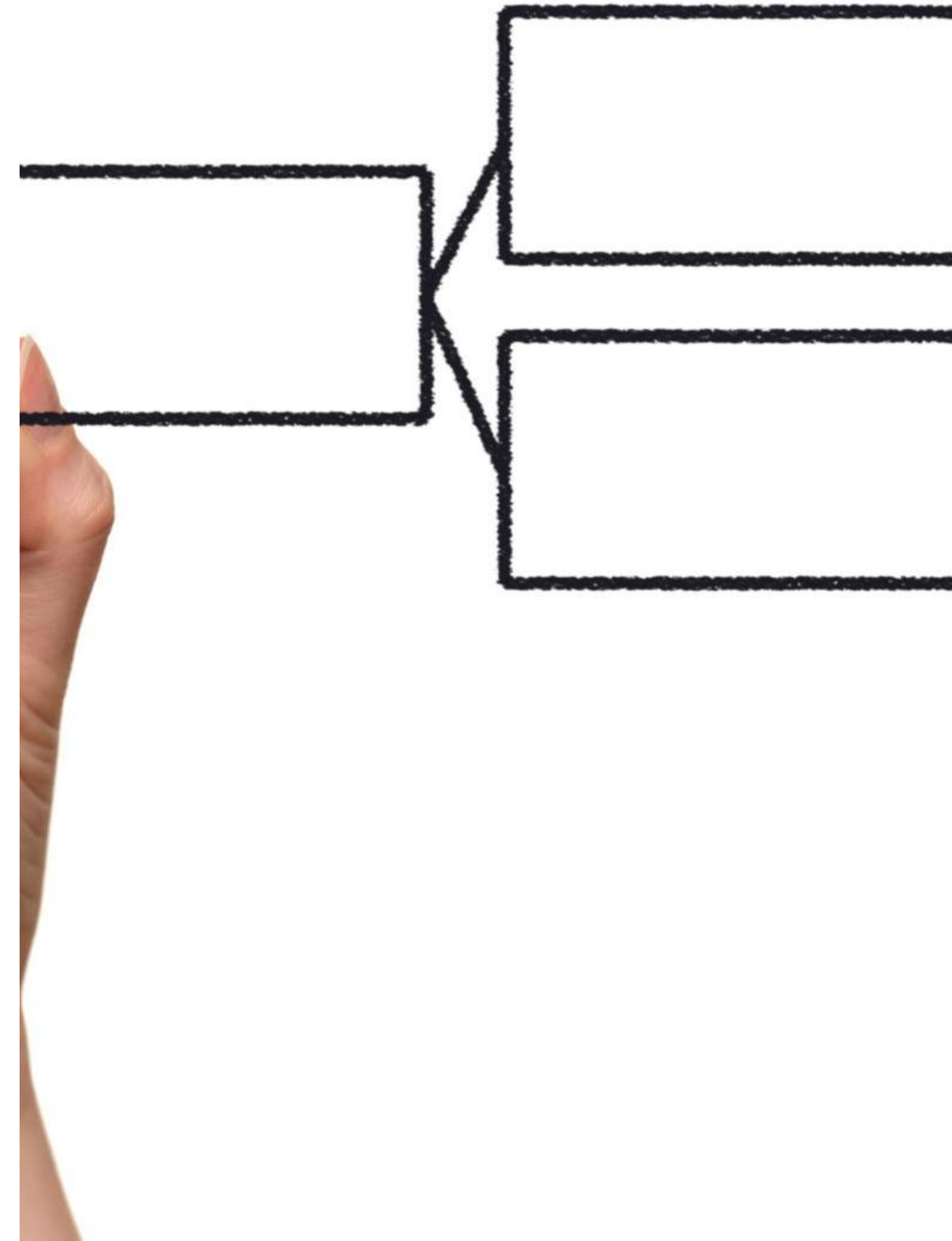
Las bases de datos de claves-valor utilizan un modelo de almacenamiento simple que se basa en pares de clave y valor, facilitando la organización de datos.

Acceso Rápido a Datos

Este modelo permite un acceso rápido a datos, lo que lo convierte en una opción ideal para aplicaciones que requieren rapidez.

Alta Disponibilidad

Las bases de datos de claves-valor son perfectas para aplicaciones que necesitan alta disponibilidad, asegurando el acceso constante a los datos.



BBDD Clave-Valor

Una base de datos clave-valor es el modelo más simple de base de datos NoSQL. En este tipo de bases de datos, los datos se almacenan como pares **clave-valor**, donde una clave única identifica a un valor asociado. Este modelo es muy eficiente para operaciones de lectura y escritura rápidas, lo que lo hace ideal para caché y sistemas de alto rendimiento.

Funcionamiento: El acceso a los datos se realiza a través de una clave única, como si fuera un gran diccionario o mapa en programación.

Casos de uso:

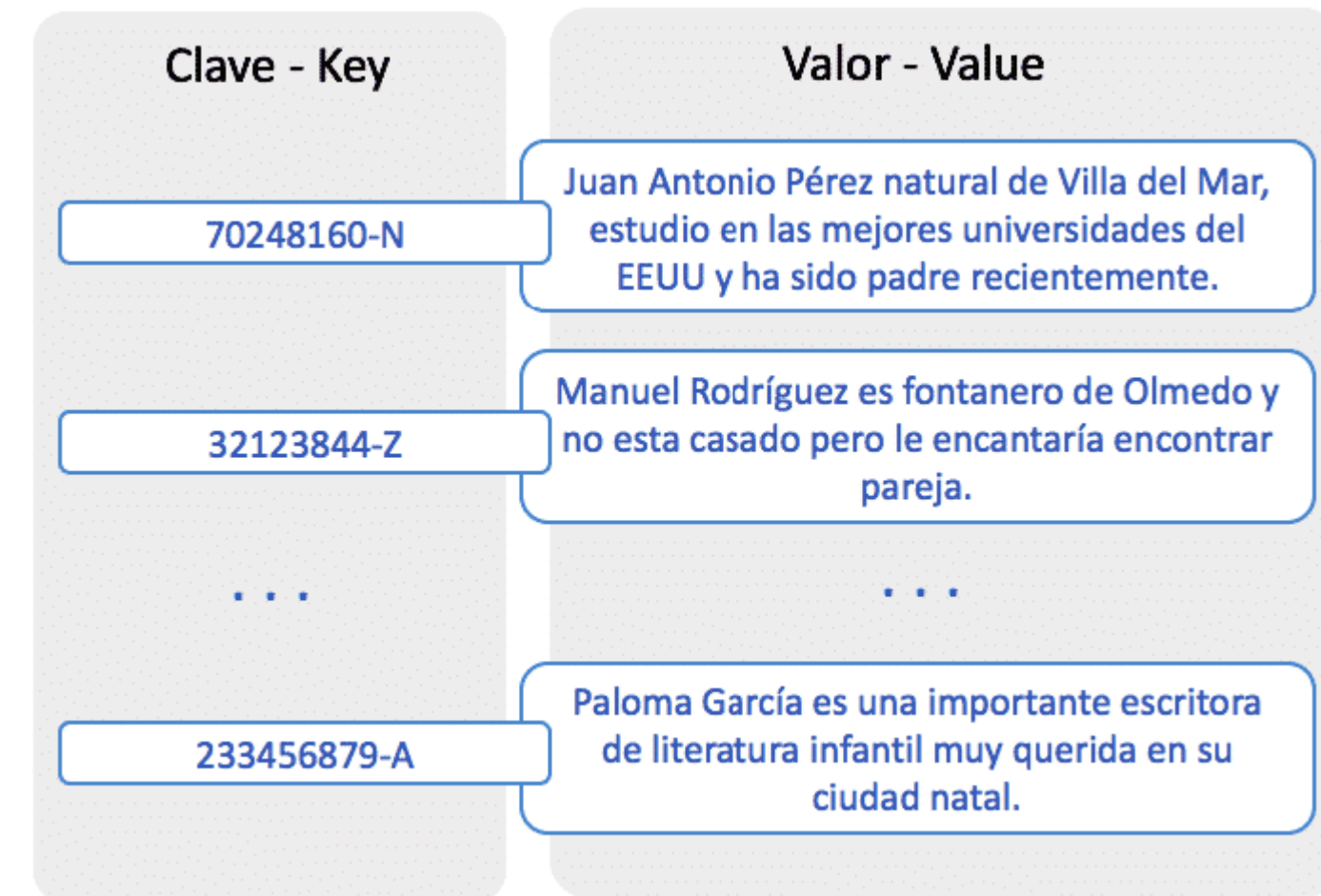
Sistemas de caché: Donde se almacenan resultados de consultas costosas para acceso rápido (ej. Redis).

Sesiones de usuario: Donde se guarda el estado del usuario en aplicaciones web.

Ejemplos:

Redis: Un sistema de base de datos en memoria que se usa principalmente para almacenar estructuras de datos en caché.

Amazon DynamoDB: Una base de datos NoSQL administrada que permite una escalabilidad masiva con baja latencia.



json

Copiar código

```
{
  "user123": "John Doe",
  "session567": "active",
  "cart789": "item1, item2, item3"
}
```

El Modelo de Datos Más Simple: Clave-Valor

Descripción:

- Almacena datos como una colección de pares clave-valor.
- Cada "clave" es un identificador único, y el "valor" es un objeto de datos (que puede ser texto, JSON, binario, etc.).
- Optimizado para operaciones rápidas de lectura y escritura basadas en la clave.

Características Clave:

- **Simplicidad:** Modelo de datos muy sencillo.
- **Rendimiento:** Extremadamente rápido para operaciones CRUD (Crear, Leer, Actualizar, Borrar) básicas.
- **Escalabilidad:** Fácil de escalar horizontalmente distribuyendo los pares clave-valor entre múltiples nodos.
- **Flexibilidad:** Los "valores" pueden tener cualquier formato.

Casos de Uso Típicos en Big Data:

- **Almacenamiento en caché:** Almacenar datos a los que se accede con frecuencia para acelerar las aplicaciones (ej., Redis, Memcached).
- **Gestión de sesiones de usuario:** Almacenar información de sesión para aplicaciones web (ej., Redis).
- **Perfiles de usuario:** Almacenar atributos y preferencias de usuarios (ej., DynamoDB).
- **Recomendaciones en tiempo real:** Recuperar rápidamente información sobre productos o contenido relacionado.
- **Ingesta de datos de alta velocidad:** Capturar datos de sensores o eventos a gran velocidad.



Clave (alumno)	Valor (nota final)
Alumno 1	9.1
Alumno 2	6.8
Alumno 3	7.3

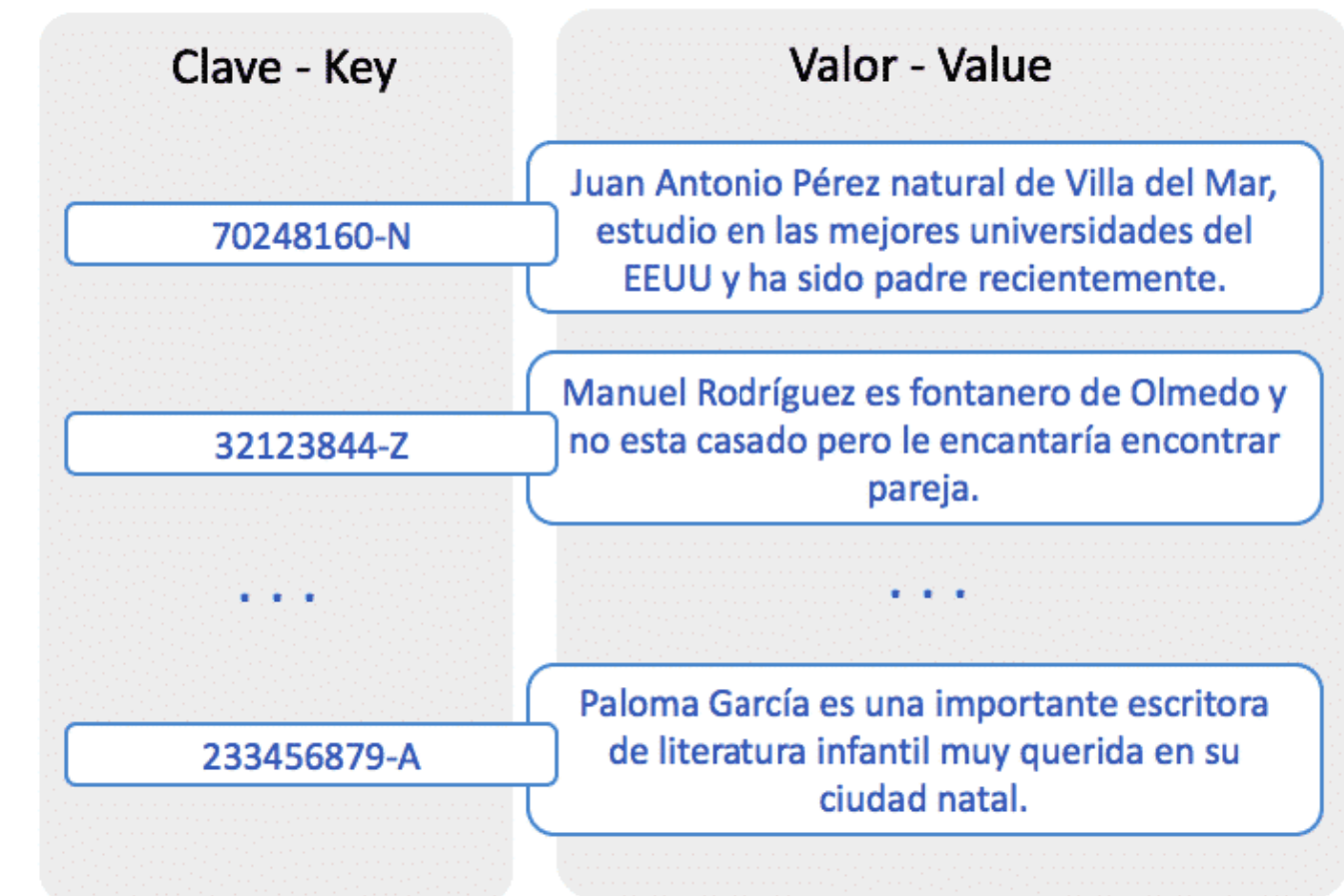
El Modelo de Datos Más Simple: Clave-Valor

Ventajas:

- Rendimiento extremadamente rápido para lecturas y escrituras simples.
- Escalabilidad horizontal sencilla.
- Diseño simple y fácil de entender.
- Alta disponibilidad.

Desventajas:

- Consultas muy limitadas más allá de la recuperación por clave.
- Falta de soporte para transacciones ACID complejas (Atomicidad, Consistencia, Aislamiento, Durabilidad) en muchos sistemas.
- Las relaciones entre los datos deben gestionarse en la aplicación.



Bases de datos de columnas

Almacenamiento en Columnas

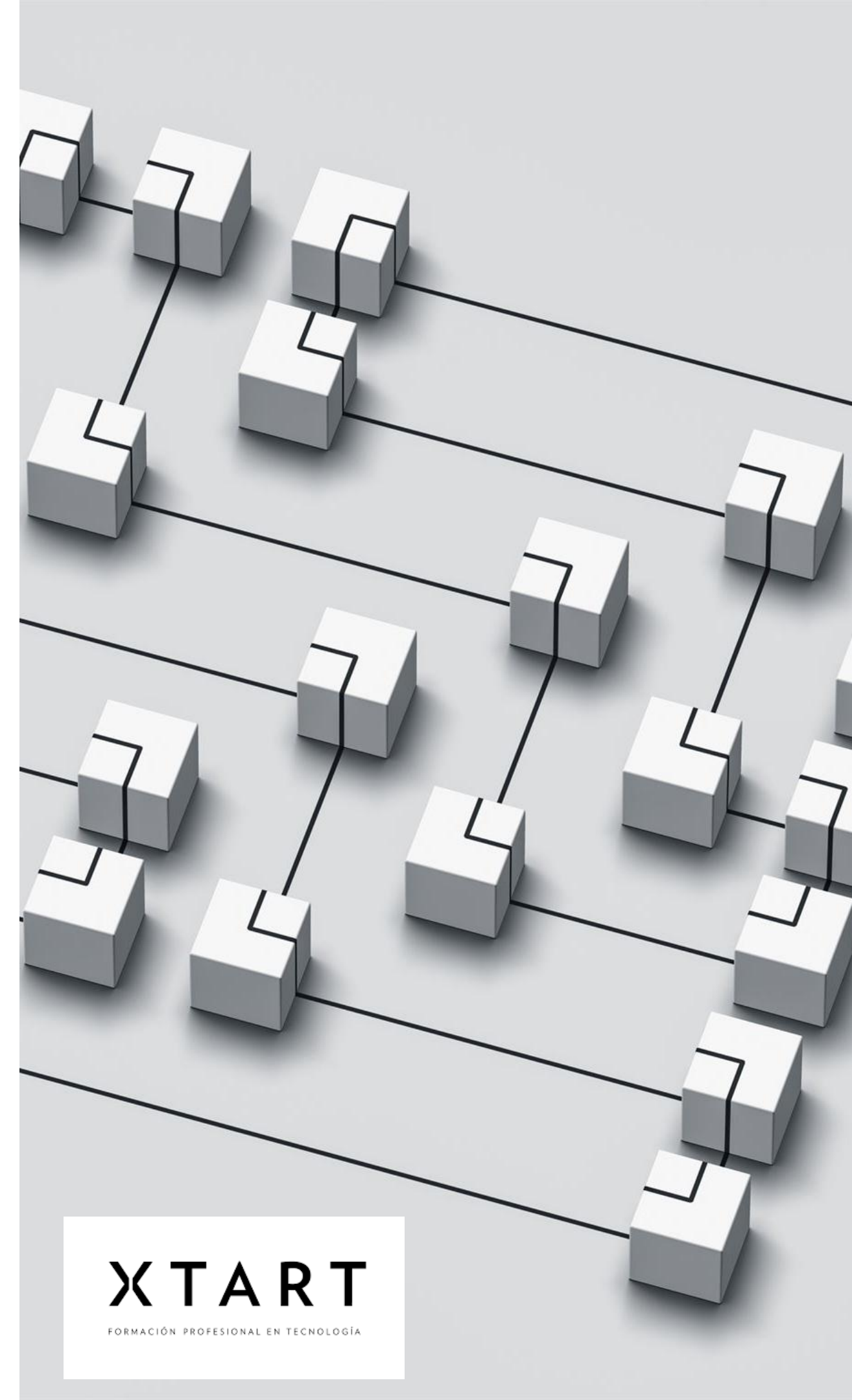
Las bases de datos de columnas organizan los datos en columnas, lo que mejora la eficiencia del almacenamiento y facilita el acceso a los datos relevantes.

Optimización de Consultas

Este tipo de base de datos es ideal para consultas analíticas, permitiendo un procesamiento más rápido y eficiente de grandes volúmenes de datos.

Manejo de Grandes Volúmenes de Datos

Son especialmente efectivas en aplicaciones que requieren el manejo de grandes conjuntos de datos, como análisis de datos y minería de datos.



Las bases de datos columnar, también conocidas como **almacenes de columnas**, almacenan los datos en columnas en lugar de filas. Este tipo de bases de datos es eficiente para realizar consultas en grandes volúmenes de datos y se usa comúnmente en aplicaciones analíticas y de Big Data, donde las consultas requieren leer muchas columnas a la vez.

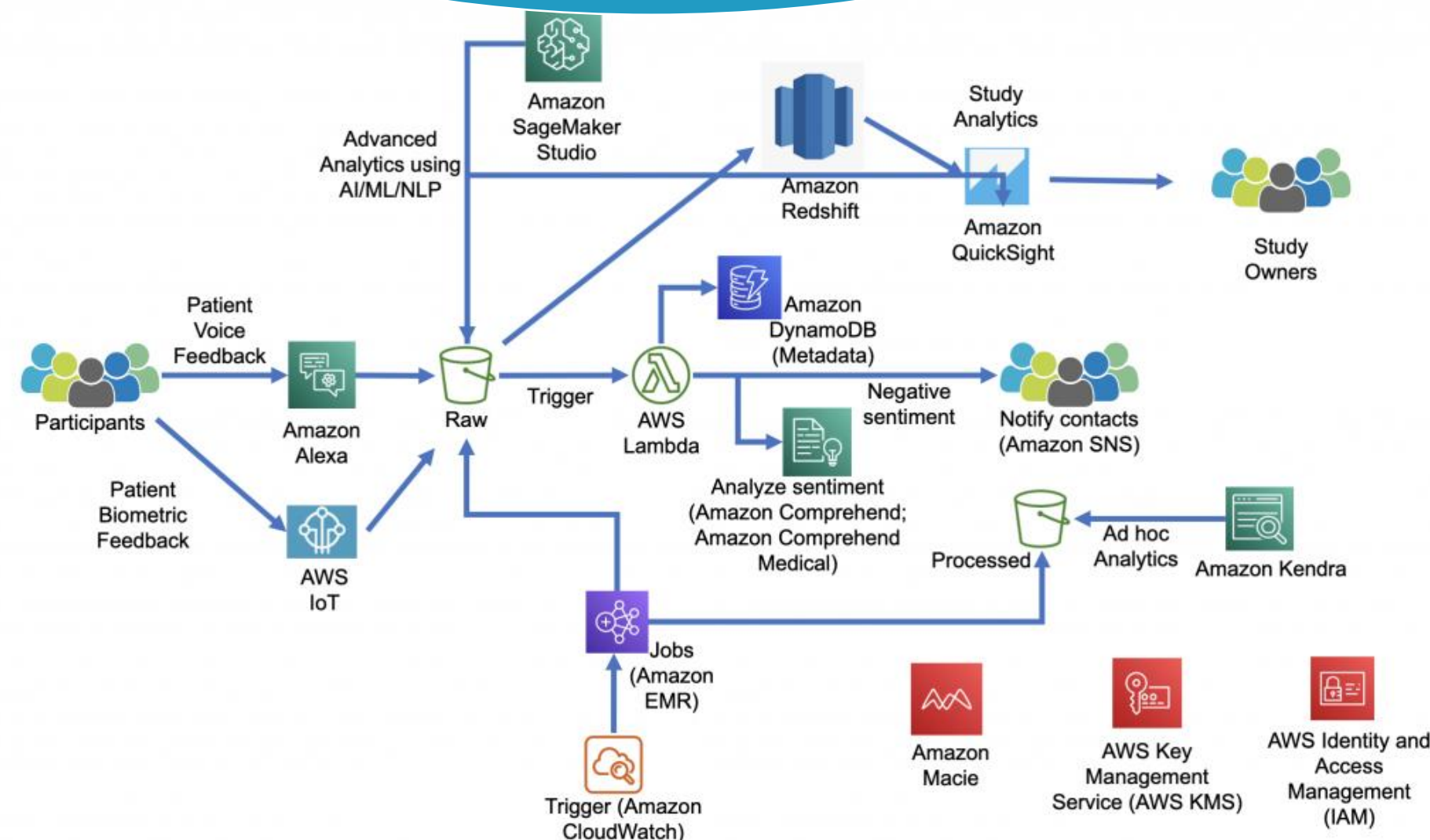
Funcionamiento: Los datos se agrupan por columnas en lugar de por filas, lo que permite leer solo las columnas necesarias para una consulta, mejorando el rendimiento en operaciones de lectura intensiva.

Casos de uso:

- **Analítica y Big Data:** En sistemas que manejan grandes volúmenes de datos y requieren realizar análisis rápido de ciertos campos (ej. **Hadoop HBase**).
- **Sistemas de recomendación:** Donde es necesario procesar grandes volúmenes de datos y buscar correlaciones rápidamente.

Ejemplos:

- **Apache Cassandra:** Optimizada para manejar grandes cantidades de datos a través de muchos servidores.
- **HBase:** Una base de datos distribuida construida sobre Hadoop
- **Amazon Redshift** es un servicio de **almacenamiento de datos en la nube** diseñado y administrado por Amazon Web Services (AWS).



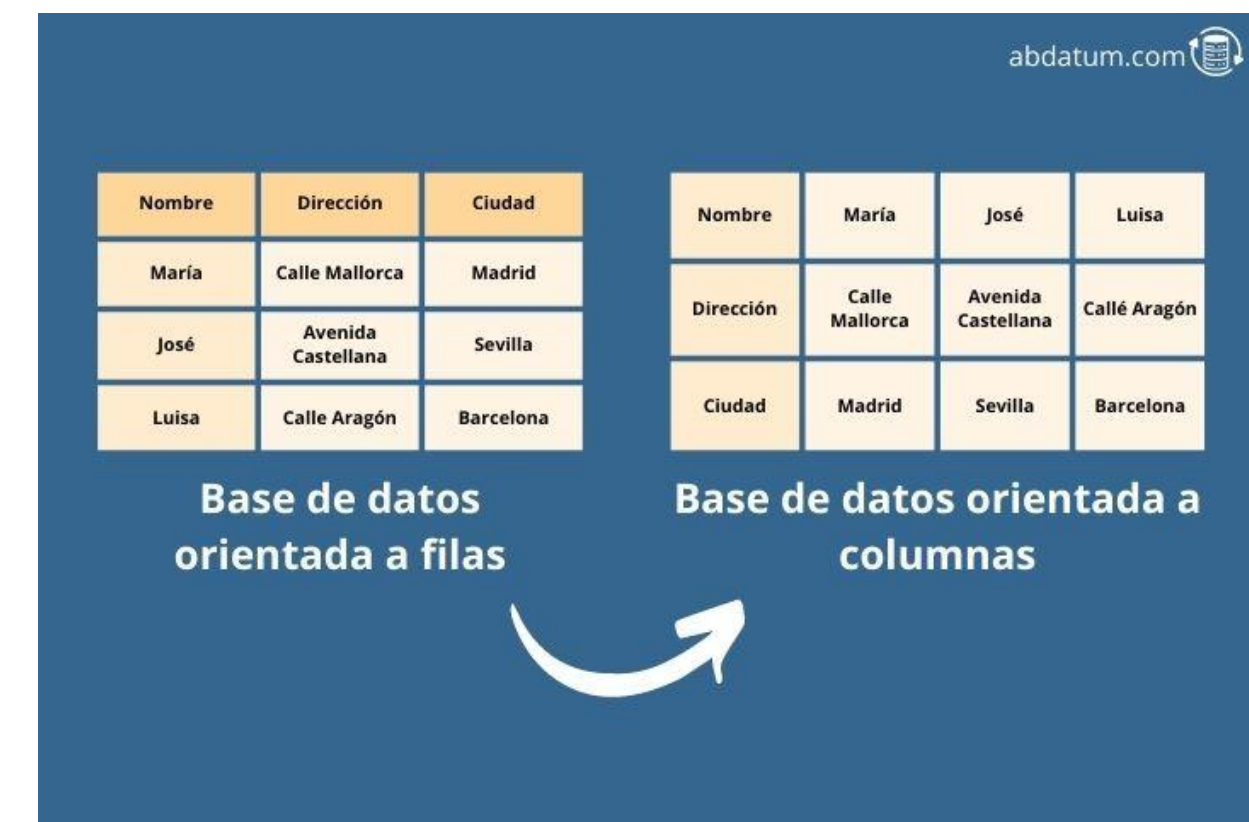
Bases de datos de columnas

Descripción:

- Almacena datos en columnas, que se agrupan en "familias de columnas".
- A diferencia de las bases de datos relacionales, las filas pueden tener diferentes columnas.
- Diseñado para manejar grandes volúmenes de datos y consultas analíticas.

Características Clave:

- **Almacenamiento orientado a columnas:** Los datos se almacenan por columna, lo que permite una lectura eficiente de columnas específicas.
- **Alta escalabilidad:** Puede escalar a petabytes de datos y miles de nodos.
- **Columnas dispersas:** Las filas no necesitan tener todas las columnas, lo que ahorra espacio de almacenamiento.
- **Claves de fila:** Se utiliza una clave de fila para identificar cada fila de datos.
- **Control de versiones:** Puede almacenar múltiples versiones de una columna.
- **Casos de Uso Típicos en Big Data:**
 - **Series de tiempo:** Almacenar datos de series de tiempo, como datos bursátiles o datos de sensores.
 - **Registros de eventos:** Almacenar grandes volúmenes de registros de eventos para análisis.
 - **Datos de sensores:** Recopilación y análisis de datos de sensores de dispositivos IoT.
 - **Personalización:** Almacenar preferencias de usuario y datos de comportamiento.
 - **Análisis:** Consultas analíticas sobre grandes conjuntos de datos.



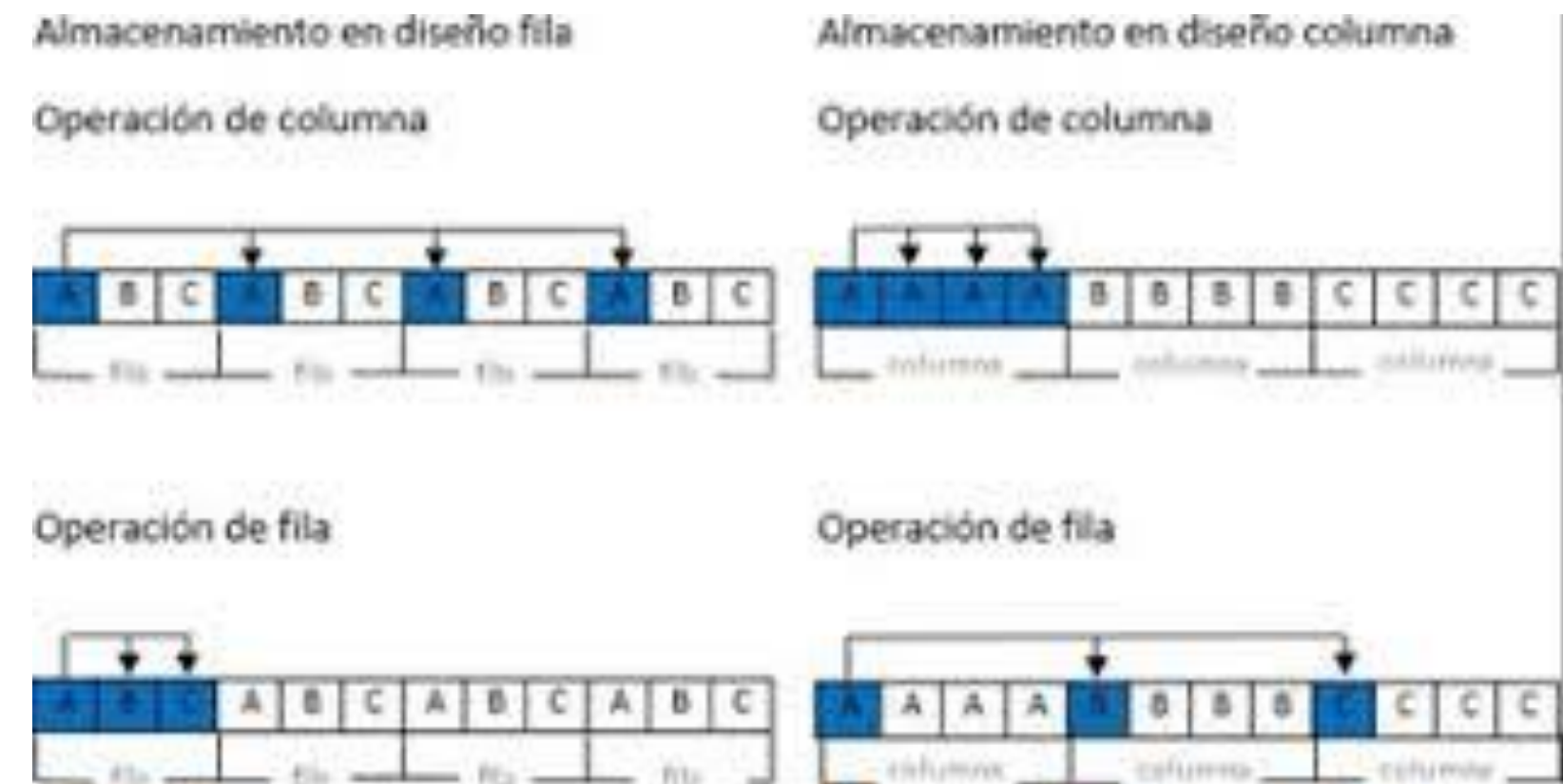
Bases de datos de columnas

Ventajas:

- Altamente escalable y distribuida.
- Almacenamiento y recuperación eficientes de columnas específicas.
- Puede manejar grandes volúmenes de datos con muchas columnas.
- Flexibilidad en el esquema.

Desventajas:

- El modelo de datos puede ser menos intuitivo que otros modelos NoSQL.
- La gestión de esquemas puede ser compleja.
- No es la mejor opción para transacciones OLTP complejas.



Bases de datos de grafos

Estructura de Grafos

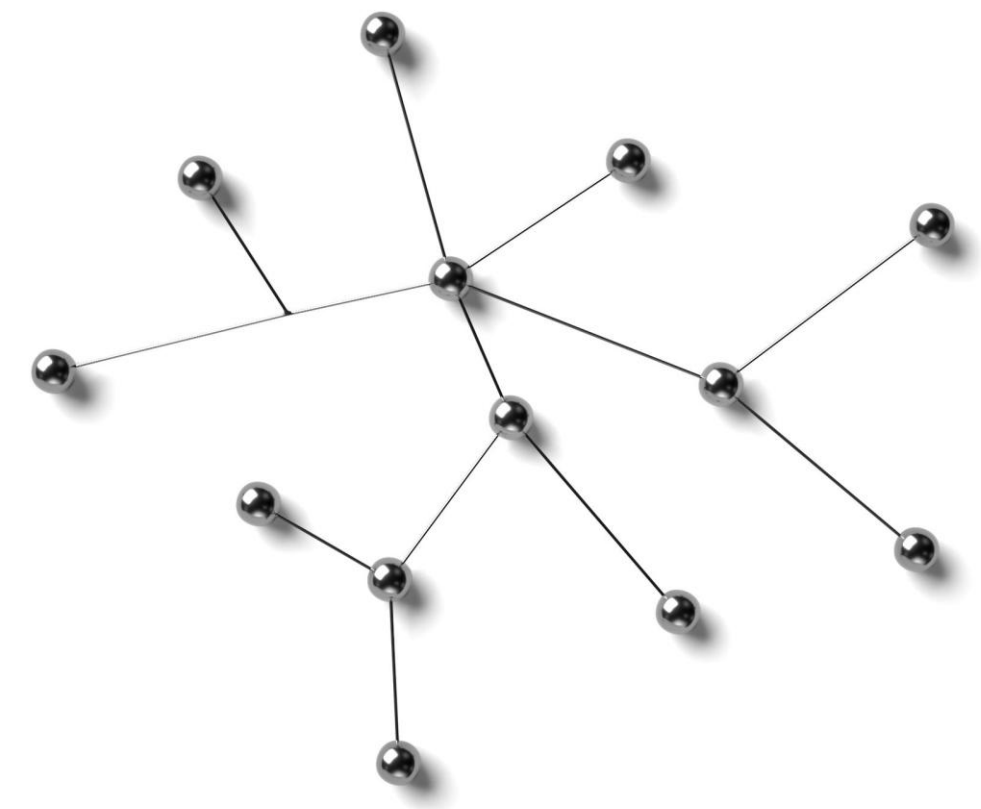
Las bases de datos de grafos utilizan una estructura de nodos y aristas para representar datos y sus relaciones de manera eficiente.

Consultas en Tiempo Real

Permiten realizar consultas en tiempo real sobre relaciones complejas, mejorando la eficiencia en la extracción de información.

Aplicaciones Prácticas

Son ideales para aplicaciones en redes sociales, análisis de relaciones y recomendación de contenido basado en conexiones.



Bases de datos de grafos

Descripción:

- Almacena datos como "nodos" (entidades) y "aristas" (relaciones) para representar las relaciones entre los datos.
- Optimizado para consultar y analizar relaciones complejas.

Características Clave:

- **Nodos y aristas:** Los datos se representan como nodos y las relaciones como aristas.
- **Relaciones con tipo:** Las aristas pueden tener tipos y propiedades.
- **Consultas de grafos:** Utiliza lenguajes de consulta especializados (ej., Cypher, Gremlin) para recorrer el grafo.
- **Indexación:** Soporta la creación de índices para optimizar las consultas.

Casos de Uso Típicos en Big Data:

- **Redes sociales:** Analizar conexiones entre usuarios, detectar comunidades.
- **Motores de recomendación:** Recomendar productos o contenido basado en las relaciones entre usuarios y artículos.
- **Detección de fraude:** Identificar patrones sospechosos en transacciones o redes.
- **Gestión del conocimiento:** Almacenar y consultar conocimiento estructurado.
- **Análisis de redes:** Analizar redes de comunicación, redes de transporte, etc.
- **Búsqueda de rutas:** Encontrar el camino más corto o más eficiente entre dos puntos.



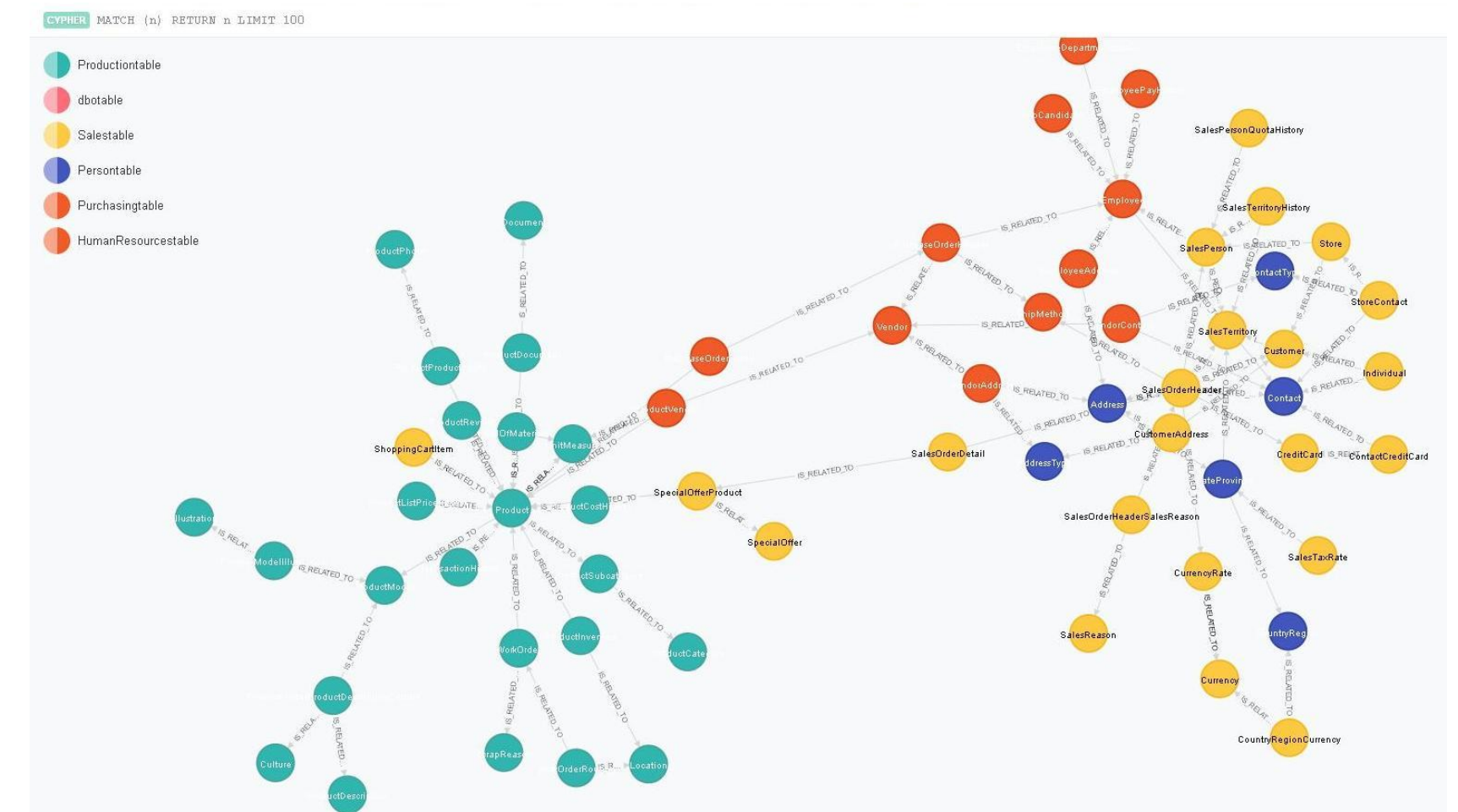
Bases de datos de grafos

Ventajas:

- Excelente para consultar relaciones complejas.
- Visualización intuitiva de datos conectados.
- Rendimiento eficiente para traversals de grafos.
- Modelo de datos flexible y expresivo.

Desventajas:

- Puede ser menos eficiente para consultas que no involucran relaciones de grafos.
- La escalabilidad puede ser un desafío para grafos muy grandes y densos.
- Los lenguajes de consulta pueden ser diferentes de SQL, lo que requiere un nuevo aprendizaje.



Las bases de datos de grafos están diseñadas para almacenar relaciones entre entidades. Los datos se organizan en nodos (entidades) y aristas (relaciones), y este enfoque es ideal para representar y consultar redes complejas, como redes sociales o mapas de tráfico.

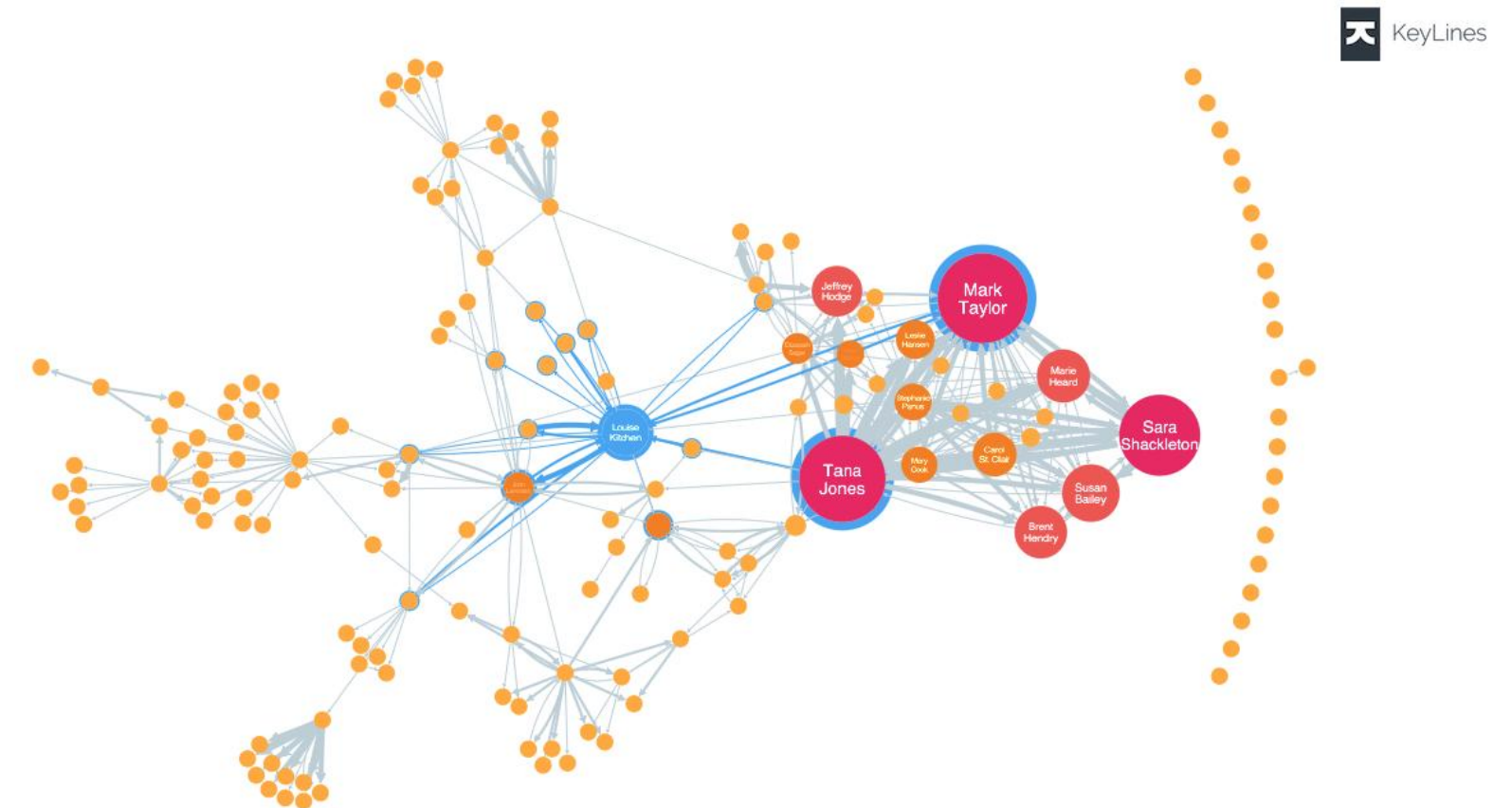
Funcionamiento: Utilizan un modelo de grafos, donde los nodos representan entidades (personas, lugares, objetos) y las aristas representan relaciones entre ellas. Esto permite realizar consultas complejas sobre las relaciones de los datos de manera eficiente.

Casos de uso:

- **Redes sociales:** Donde las relaciones entre personas (amistades, seguidores, etc.) son fundamentales.
- **Recomendaciones:** Como en los sistemas de recomendación de productos o amigos, que necesitan encontrar patrones en las relaciones.
- **Mapas y rutas:** Para aplicaciones que necesitan realizar cálculos complejos de rutas o conectividad.

Ejemplos:

- **Neo4j:** Es una de las bases de datos de grafos más utilizadas, popular por su capacidad para realizar consultas sobre redes complejas.
- **Amazon Neptune:** Un servicio de base de datos de grafos completamente gestionado.



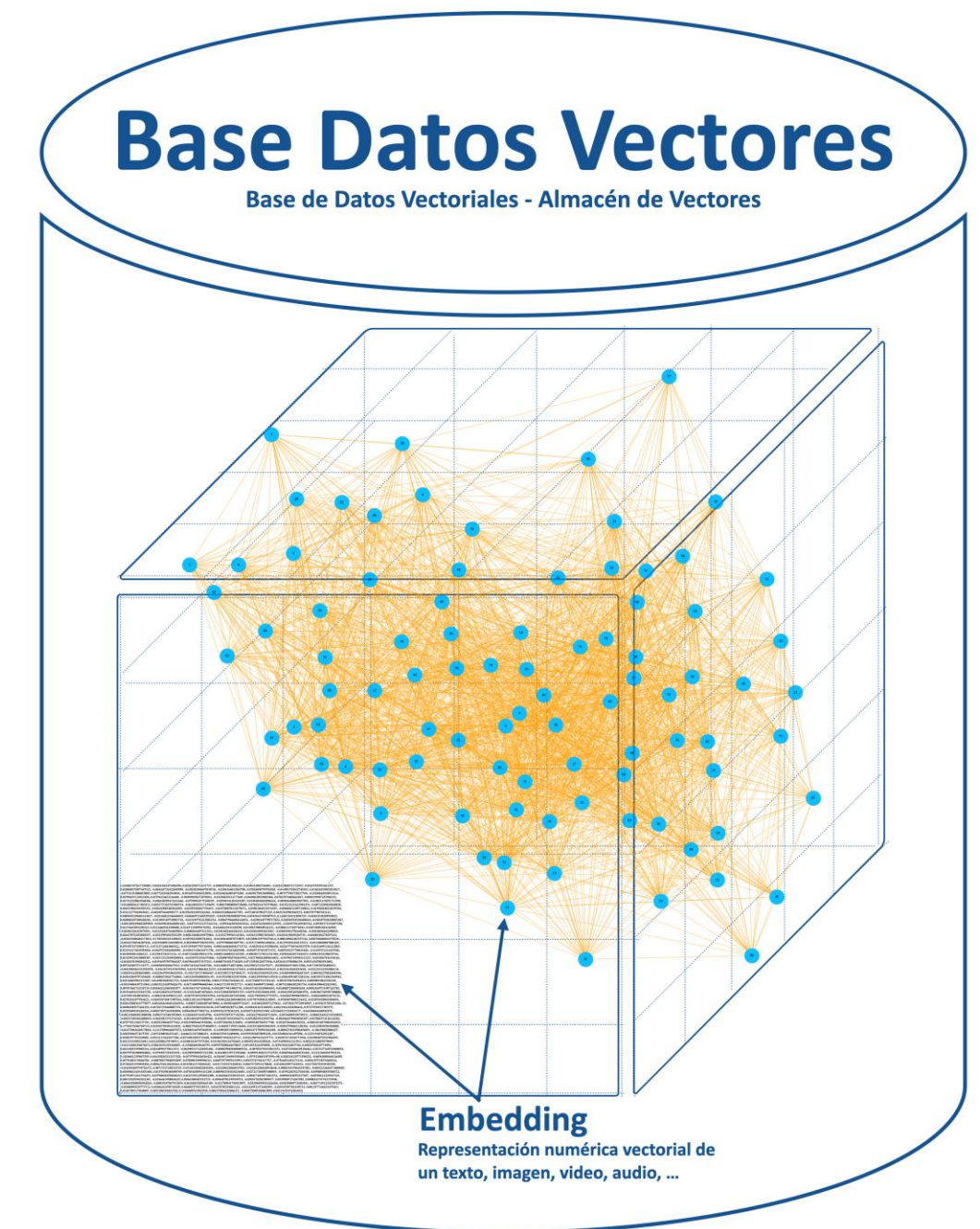
Las **bases de datos vectoriales** son un tipo especializado de base de datos NoSQL diseñadas específicamente para almacenar, indexar y consultar **embeddings vectoriales**.

¿Qué son los embeddings vectoriales?

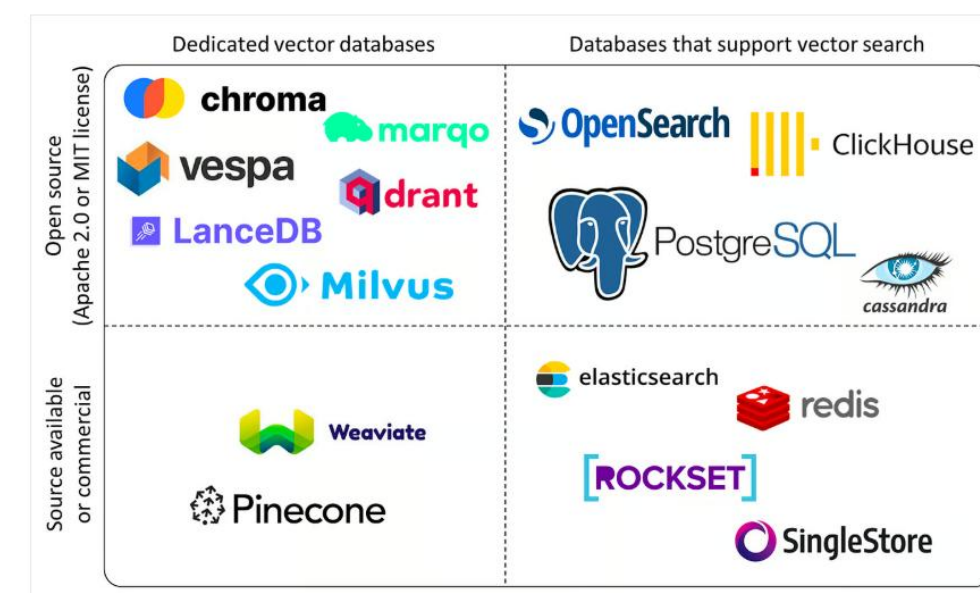
Los embeddings vectoriales son representaciones numéricas de alta dimensión de datos (texto, imágenes, audio, video, etc.) que capturan el significado semántico y características de esos datos. Los elementos similares tienen vectores cercanos en el espacio vectorial.

Características clave de las bases de datos vectoriales NoSQL:

- **Almacenamiento de vectores de alta dimensión:** Optimizadas para manejar con cientos o incluso miles de dimensiones.
- **Indexación eficiente para búsqueda de similitud:** Implementan algoritmos de especializados (como HNSW, Annoy, Faiss) para realizar búsquedas rápidas y escalables de los vectores más similares a un vector de consulta.
- **Consulta por similitud:** La principal forma de consulta es encontrar los vecinos cercanos (Nearest Neighbors - NN) a un vector dado, lo que permite realizar búsquedas semánticas o basadas en la similitud de características.
- **Escalabilidad horizontal:** Diseñadas para escalar horizontalmente y manejar volúmenes de vectores.
- **Integración con flujos de trabajo de IA/ML:** Se integran fácilmente con bibliotecas y frameworks de machine learning para el almacenamiento y recuperación de embeddings generados por modelos de IA.



- Existen varias bases de datos NoSQL que se clasifican como bases de datos vectoriales, algunas de ellas son
- **Pinecone**: Una base de datos vectorial **completamente gestionada** en la nube, diseñada específicamente para aplicaciones de IA y machine learning.
- **Weaviate**: Una base de datos vectorial **de código abierto** que ofrece capacidades de búsqueda semántica y modelado de conocimiento.
- **Qdrant**: Otra base de datos vectorial **de código abierto** construida sobre Rust, enfocada en la velocidad y la eficiencia.
- **Milvus**: Una base de datos vectorial **de código abierto** que soporta una amplia variedad de métricas de distancia e índices.
- **ChromaDB**: Una base de datos de embeddings **de código abierto** diseñada para ser fácil de usar e integrar con frameworks de LLMs.
- **Faiss (Facebook AI Similarity Search)**: Una **biblioteca** para una búsqueda de similitud eficiente y clustering de vectores densos. Aunque no es una base de datos completa en sí misma, es una tecnología subyacente utilizada por muchas bases de datos vectoriales.
- **Annoy (Approximate Nearest Neighbors Oh Yeah)**: Otra **biblioteca** de código abierto para la búsqueda aproximada de vecinos más cercanos.
- **Azure Cosmos DB con capacidades de búsqueda vectorial**: Azure ha integrado funcionalidades de base de datos vectorial en su API para NoSQL y para PostgreSQL.
- **MongoDB Atlas Vector Search**: MongoDB ha añadido capacidades de búsqueda vectorial a su servicio en la nube Atlas.



El panorama de las bases de datos vectoriales (fuente de la imagen)

Ejemplo:

1. Textos y sus Vectores (Simplificados):

Imaginemos que tenemos los siguientes textos y sus representaciones vectoriales simplificadas (en la realidad, los vectores tendrían muchas más dimensiones)

Texto	Vector (Representación Simplificada)
El gato duerme en la alfombra	[0.8, 0.2, 0.5, 0.1]
Un felino está durmiendo	[0.7, 0.3, 0.4, 0.2]
El perro juega en el parque	[0.1, 0.9, 0.3, 0.6]
Un can corre felizmente	[0.2, 0.8, 0.4, 0.5]

En la práctica, estos vectores se generan mediante modelos de **Machine Learning** llamados **modelos de embedding** (por ejemplo, Word2Vec, GloVe, Sentence-BERT para texto). Estos modelos han sido entrenados en grandes cantidades de datos para aprender a capturar el significado de las palabras y las frases en forma numérica.

•**Funcionamiento:** Las dimensiones del vector podrían representar características abstractas como "felino", "acción de dormir", "lugar interior", "acción de jugar", "lugar exterior", etc. Los valores en el vector indican cuánto de esa característica está presente en el texto

Si queremos buscar textos similares a "**gato somnoliento**", primero generaríamos un vector para esta frase (usando el mismo modelo de embedding):

Texto de Búsqueda	Vector (Generado)
gato somnoliento	[0.75, 0.25, 0.45, 0.15]

En la practica, estos vectores se generan mediante modelos de **Machine Learning** llamados **modelos de embedding** (por ejemplo, Word2Vec, GloVe, Sentence-BERT para texto). Estos modelos han sido entrenados en grandes cantidades de datos para aprender a capturar el significado de las palabras y las frases en forma numérica.

•**Idea Intuitiva:** Las dimensiones del vector podrían representar características abstractas como "felino", "acción de dormir", "lugar interior", "acción de jugar", "lugar exterior", etc. Los valores en el vector indican cuánto de esa característica está presente en el texto.

Búsqueda de Texto:

Si queremos buscar textos similares a "**gato somnoliento**", primero generaríamos un vector para esta frase (usando el mismo modelo de embedding):

Luego, calcularíamos la "distancia" o "similitud" entre este vector de búsqueda y los vectores de los textos en nuestra base de datos vectorial. Una métrica común es la **similitud del coseno**. Cuanto menor sea la distancia (o mayor la similitud del coseno), más similares serán los textos.

En este ejemplo simplificado, el vector de "gato somnoliento" estaría más cerca de los vectores de "El gato duerme en la alfombra" y "Un felino está durmiendo" que de los textos sobre perros

Servicios de bases de datos NoSQL en Amazon AWS

Amazon REDSHIFT

¿Qué es Amazon Redshift?

- Un servicio de **almacén de datos en la nube** completamente administrado que permite analizar grandes volúmenes de datos de forma rápida y eficiente.
- Optimizado para **consultas analíticas** y **business intelligence (BI)**.
- Utiliza almacenamiento en **formato de columnas** para mejorar el rendimiento de las consultas.
- **Características Clave:**
- **Arquitectura de procesamiento paralelo masivo (MPP):** Distribuye el procesamiento de datos y las consultas entre múltiples nodos para un rendimiento rápido.
- **Almacenamiento en columnas:** Almacena los datos por columnas en lugar de filas, lo que reduce la cantidad de E/S de disco necesaria para las consultas analíticas.
- **Escalabilidad:** Se escala fácilmente para manejar petabytes de datos.
- **Rendimiento optimizado:** Ofrece varias características para optimizar el rendimiento de las consultas, como la compilación de consultas, el almacenamiento en caché y la optimización del almacenamiento.
- **Seguridad:** Proporciona características de seguridad integrales, que incluyen cifrado de datos en tránsito y en reposo, control de acceso y auditoría.
- **Integración con el ecosistema de AWS:** Se integra a la perfección con otros servicios de AWS, como Amazon S3, Amazon EMR y Amazon QuickSight.

Casos de Uso Comunes:

- **Business intelligence (BI):** Análisis de datos para obtener información y respaldar la toma de decisiones.
- **Análisis de datos:** Ejecución de consultas complejas en grandes conjuntos de datos.
- **Data warehousing:** Consolidación de datos de diversas fuentes en un repositorio central para la generación de informes y el análisis.
- **Paneles de control:** Creación de paneles interactivos para visualizar datos y métricas clave.
- **Análisis predictivo:** Uso de datos históricos para predecir resultados futuros y tendencias.



Amazon DynamoDB: Clave-valor y documentos

Servicio de Base de Datos NoSQL

DynamoDB es un servicio de base de datos NoSQL totalmente gestionado, ideal para aplicaciones que requieren flexibilidad y escalabilidad.

Escalabilidad Automática

Proporciona escalabilidad automática, permitiendo ajustar los recursos según la demanda de la aplicación sin interrupciones.

Alto Rendimiento y Seguridad

DynamoDB garantiza alto rendimiento y características de seguridad robustas, protegiendo los datos críticos de las aplicaciones modernas.



Amazon Neptune: Base de datos de grafos

Servicio de Base de Datos de Grafos

Amazon Neptune es un servicio diseñado específicamente para el manejo de bases de datos de grafos, optimizando la creación y consulta de aplicaciones relacionales.

Soporte para Modelos de Grafos

El servicio soporta modelos de grafos populares como Apache TinkerPop y RDF, facilitando el análisis de datos complejos.

Análisis de Conexiones de Datos

Neptune permite un análisis profundo de las conexiones entre los datos, proporcionando información valiosa para la toma de decisiones.

Amazon Keyspaces: Columnas

Compatibilidad con Apache Cassandra

Amazon Keyspaces es completamente compatible con Apache Cassandra, lo que permite a los desarrolladores migrar y escalar sus aplicaciones fácilmente.

Escalabilidad Horizontal

El servicio ofrece escalabilidad horizontal, permitiendo a las organizaciones manejar un volumen creciente de datos sin sacrificar el rendimiento.

Baja Latencia

Amazon Keyspaces asegura baja latencia en las operaciones de base de datos, optimizando la experiencia del usuario final en aplicaciones críticas.



Casos de uso de aplicación de bases de datos NoSQL

Aplicaciones web y móviles

Ventajas de NoSQL

Las bases de datos NoSQL ofrecen un rendimiento optimizado para aplicaciones que requieren manejar grandes volúmenes de datos de manera eficiente.

Flexibilidad en el Desarrollo

Las bases de datos NoSQL permiten a los desarrolladores adaptarse rápidamente a los cambios en los requisitos del proyecto gracias a su estructura flexible.

Respuestas Rápidas

La capacidad de respuesta rápida de las bases de datos NoSQL es fundamental para aplicaciones web y móviles que requieren interacción en tiempo real.



Big Data y análisis en tiempo real

Uso de bases de datos NoSQL

Las bases de datos NoSQL son esenciales en entornos de Big Data, permitiendo una gestión eficiente de datos grandes y complejos.

Procesamiento en tiempo real

El procesamiento de datos en tiempo real es crucial para la toma de decisiones rápidas y efectivas en el análisis de Big Data.

Manejo de datos no estructurados

La capacidad de manejar datos no estructurados en bases de datos NoSQL es fundamental para gestionar la variedad de datos generados hoy en día.



Redes sociales y gestión de relaciones

Bases de datos NoSQL

Las redes sociales utilizan bases de datos NoSQL para gestionar relaciones complejas entre usuarios debido a su flexibilidad y escalabilidad.

Gestión de Relaciones

La gestión de relaciones en redes sociales es crucial para entender la interacción entre usuarios y mejorar la experiencia del usuario.

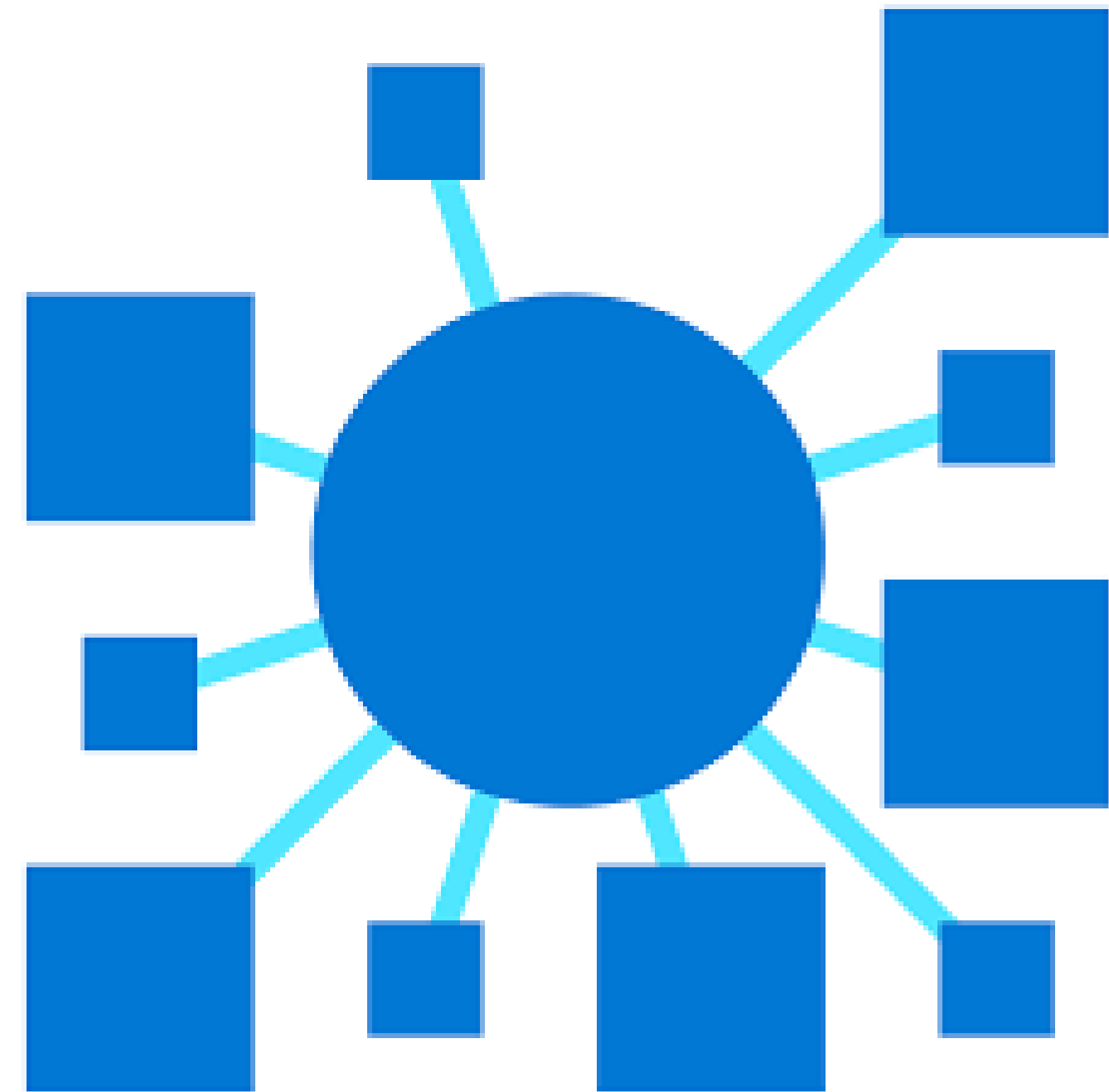
Consultas Eficientes

Las bases de datos NoSQL permiten realizar consultas rápidas y eficientes, facilitando el acceso a datos de usuarios y conexiones.

BBDD NoSQL e IA

¿Por qué son importantes las bases de datos NoSQL en la IA?

- **Flexibilidad de esquema:** Los modelos de datos en IA a menudo evolucionan rápidamente y pueden incluir datos no estructurados o semiestructurados (texto, imágenes, audio, video). Las bases de datos NoSQL, con esquemas dinámicos o sin esquema, permiten almacenar y gestionar estos datos de manera más flexible que las bases de datos relacionales tradicionales.
- **Escalabilidad horizontal:** Las aplicaciones de IA, especialmente aquellas que trabajan con grandes volúmenes de datos (Big Data), requieren una alta escalabilidad. Las bases de datos NoSQL están diseñadas para escalar horizontalmente, distribuyendo los datos y la carga de trabajo en múltiples servidores, lo que permite manejar grandes cantidades de información y tráfico.
- **Rendimiento:** Para muchas tareas de IA, como el procesamiento en tiempo real o el análisis de grandes conjuntos de datos, se requiere un alto rendimiento en lectura y escritura. Las bases de datos NoSQL están optimizadas para patrones de acceso específicos y pueden ofrecer un rendimiento superior en ciertos casos de uso.
- **Diversidad de modelos de datos:** La IA abarca una amplia gama de aplicaciones con diferentes tipos de datos y relaciones. Las bases de datos NoSQL ofrecen varios modelos de datos (documento, clave-valor, grafo, columna), lo que permite elegir el más adecuado para cada necesidad específica.



Tipos de bases de datos NoSQL relevantes para la IA:

- **Bases de datos de documentos (ej. MongoDB, Couchbase):** Almacenan datos en documentos similares a JSON. Son ideales para datos semiestructurados y ofrecen flexibilidad para manejar diferentes estructuras de datos dentro de la misma colección. Muy útiles para almacenar features de modelos de ML, metadatos de archivos no estructurados o resultados de procesamiento de lenguaje natural.
- **Bases de datos clave-valor (ej. Redis, Memcached):** Almacenan datos como pares clave-valor. Son extremadamente rápidas para operaciones de lectura y escritura, lo que las hace útiles para el almacenamiento en caché de resultados de modelos de IA, gestión de sesiones de usuarios o almacenamiento de embeddings vectoriales para búsquedas rápidas.
- **Bases de datos de grafos (ej. Neo4j):** Almacenan datos como nodos y relaciones. Son excelentes para representar y consultar datos altamente conectados, como redes sociales, sistemas de recomendación o análisis de conocimiento. En IA, se utilizan para construir grafos de conocimiento, modelar relaciones entre entidades o para sistemas de recomendación basados en grafos.
- **Bases de datos orientadas a columnas (ej. Cassandra, HBase):** Almacenan datos en columnas en lugar de filas. Son eficientes para consultas analíticas sobre grandes conjuntos de datos, lo que las hace útiles para el análisis de features a gran escala o para almacenar series de tiempo generadas por sensores en aplicaciones de IoT con IA.
- **Bases de datos vectoriales (ej. Pinecone, Milvus, ChromaDB):** Optimizadas específicamente para almacenar y buscar embeddings vectoriales, representaciones numéricas de datos (texto, imágenes, audio). Son fundamentales para aplicaciones de búsqueda semántica, sistemas de recomendación avanzados y tareas de recuperación aumentada (Retrieval-Augmented Generation - RAG) en procesamiento de lenguaje natural.

Implementación y gestión en AWS

Configuración y despliegue de bases de datos NoSQL en AWS

Uso de la consola de AWS

La consola de administración de AWS permite a los usuarios gestionar fácilmente las configuraciones de bases de datos NoSQL de forma intuitiva.

Configuraciones personalizadas

Los usuarios pueden seleccionar diferentes configuraciones según las necesidades específicas de su aplicación, optimizando el rendimiento y la escalabilidad.



Escalabilidad y rendimiento

Escalabilidad Eficiente

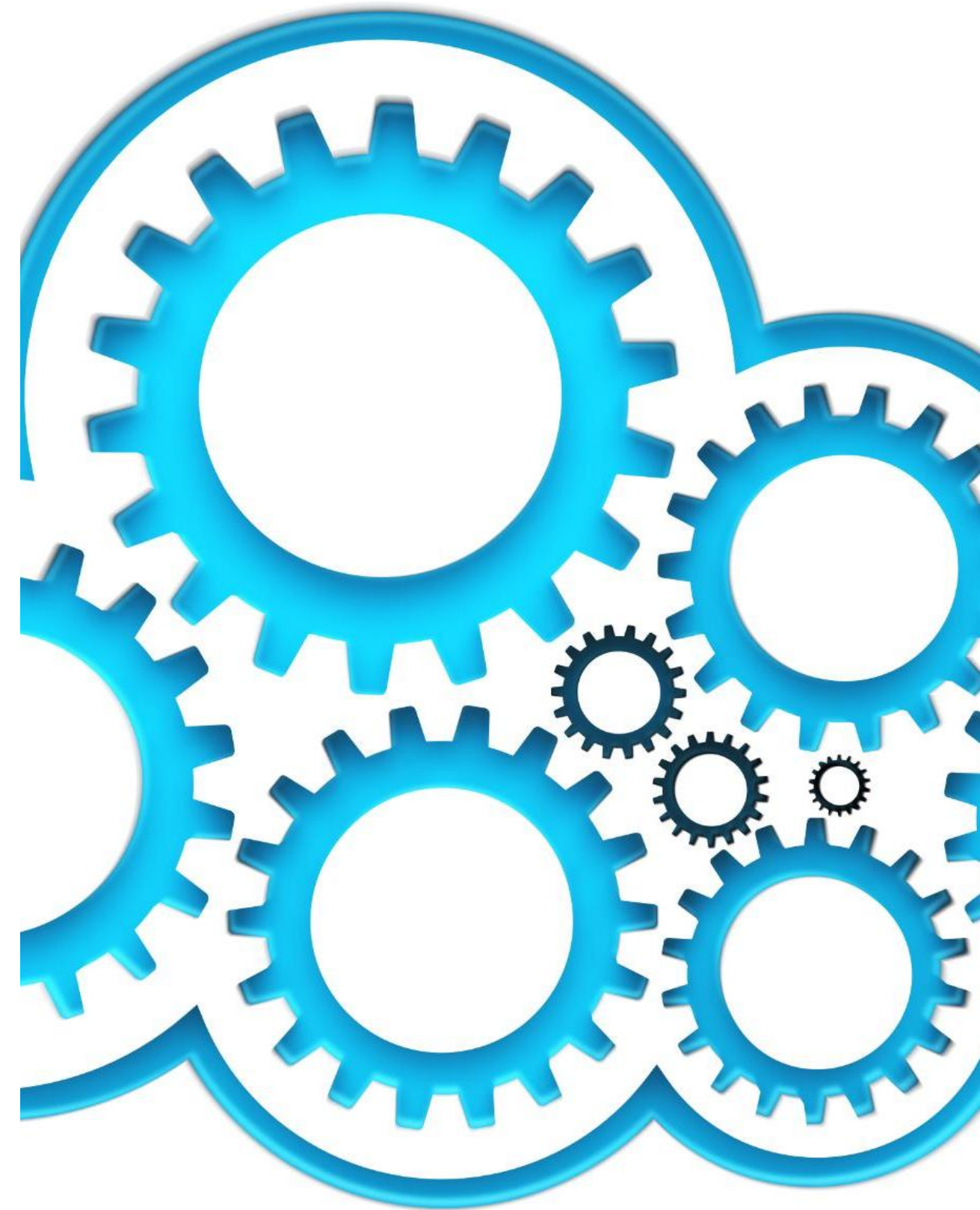
Las bases de datos NoSQL en AWS permiten escalar de manera eficiente según las demandas de la aplicación, facilitando el crecimiento.

Ajuste de Capacidad

AWS permite ajustar la capacidad de almacenamiento y rendimiento dinámicamente, asegurando un servicio óptimo según sea necesario.

Rendimiento Óptimo

Gracias a la escalabilidad, las aplicaciones pueden mantener un rendimiento óptimo incluso con un alto volumen de datos y tráfico.



Seguridad y mejores prácticas



Importancia de la Seguridad

La seguridad es crucial al trabajar con bases de datos NoSQL en la nube, protegiendo datos sensibles de accesos no autorizados.

Uso de Cifrado

Implementar cifrado es esencial para proteger los datos almacenados y en tránsito, garantizando que solo los usuarios autorizados puedan acceder a ellos.

Control de Acceso

El control de acceso adecuado ayuda a limitar quién puede ver y modificar los datos, lo que es vital para la seguridad de la información.

Auditorías Regulares

Realizar auditorías periódicas es fundamental para garantizar la conformidad con las normativas y detectar posibles vulnerabilidades.

Conclusión

Flexibilidad de NoSQL

Las bases de datos NoSQL ofrecen una flexibilidad notable para adaptarse a diferentes tipos de datos y necesidades organizativas.

Solución para Big Data

NoSQL es una solución ideal para gestionar grandes volúmenes de datos, permitiendo un procesamiento ágil y eficaz.

Innovación en AWS

Con los servicios de AWS, las organizaciones pueden integrar bases de datos NoSQL para potenciar su capacidad de innovación y mejora continua.

4/ Redshift

Redshift ML

Redshift ML, una funcionalidad de Amazon Redshift, te permite crear, entrenar e implementar modelos de aprendizaje automático utilizando comandos SQL familiares directamente dentro de tu almacén de datos. Esto une el mundo del almacenamiento de datos con el aprendizaje automático, ofreciendo varios beneficios clave:

Esto es para lo que sirve Redshift ML:

- **Integración de ML simplificada:** No necesitas ser un experto en aprendizaje automático. Redshift ML se integra con Amazon SageMaker, un servicio de ML completamente administrado, permitiéndote construir modelos usando SQL.
- **ML sobre tus datos de Redshift:** Entrena modelos de ML directamente sobre las grandes cantidades de datos que ya residen en tu almacén de datos de Redshift sin la necesidad de complejos movimientos de datos.
- **Analítica predictiva en SQL:** Una vez entrenado, el modelo de ML se convierte en una función SQL dentro de Redshift. Luego, puedes aplicar fácilmente estos modelos a tus datos utilizando consultas SQL estándar para generar predicciones.
- **Incorporación de predicciones en aplicaciones:** Integra predicciones como detección de fraude, puntuación de riesgo, predicción de abandono de clientes y más directamente en tus consultas, informes y paneles de control.
- **Construcción de modelos automatizada:** Redshift ML puede seleccionar automáticamente el mejor modelo y ajustarlo basándose en tus datos utilizando Amazon SageMaker Autopilot. También tienes la opción de especificar un algoritmo en particular.
- **Rentable:** Al llevar las capacidades de ML a Redshift, puedes reducir potencialmente los gastos generales y los costos asociados con entornos de ML y pipelines de datos separados. Principalmente pagas por los recursos de SageMaker utilizados durante el entrenamiento del modelo. La inferencia dentro de Redshift generalmente no tiene costo adicional.
- **Información más rápida:** Obtén predicciones e intégralas en tus flujos de trabajo de análisis más rápidamente, lo que lleva a una toma de decisiones más rápida e informada.
- **Mayor eficiencia:** Automatiza tareas repetitivas relacionadas con la preparación de datos y la implementación de modelos dentro del entorno familiar de Redshift.
- **Gestión de datos mejorada:** Redshift ML contribuye a una mejor gestión de datos al habilitar el aprendizaje automático dentro de la base de datos, reduciendo la necesidad de mover y administrar datos entre diferentes sistemas.

Cuándo utilizar Redshift ML:

- **Exploración rápida y creación de prototipos:** Cuando un científico de datos necesita construir rápidamente un modelo básico para entender la viabilidad de una idea o para obtener información inicial de los datos almacenados en Redshift. La facilidad de usar SQL para esto puede acelerar el proceso inicial.
- **Modelos sencillos y bien definidos:** Para problemas de aprendizaje automático estándar como clasificación binaria, regresión o clustering, donde los algoritmos automatizados de SageMaker Autopilot pueden ser suficientes.
- **Integración directa con datos de Redshift:** Cuando el conjunto de datos principal para el entrenamiento ya reside en Redshift y mover grandes cantidades de datos a otra plataforma sería ineficiente o costoso.
- **Operacionalización sencilla dentro de Redshift:** Si el objetivo principal es integrar las predicciones directamente en consultas SQL, informes o dashboards dentro del entorno de Redshift. La función SQL generada facilita esta integración.
- **Colaboración con analistas de datos:** Redshift ML puede facilitar la colaboración entre científicos de datos y analistas de datos, ya que estos últimos pueden utilizar los modelos creados a través de SQL sin necesidad de conocimientos profundos de ML.
- **Casos de uso donde la latencia no es crítica:** La inferencia con modelos de Redshift ML se realiza dentro del clúster de Redshift, lo cual puede tener una latencia mayor en comparación con endpoints de inferencia dedicados en SageMaker.

Cuándo NO utilizar Redshift ML:

- **Modelos complejos y personalizados:** Para algoritmos de aprendizaje profundo, modelos que requieren arquitecturas de red neuronal específicas o técnicas de ML avanzadas que no están soportadas por SageMaker Autopilot directamente a través de Redshift ML.
- **Necesidad de un control granular del proceso de ML:** Los científicos de datos que requieren un control total sobre la ingeniería de features, la selección de modelos, el ajuste de hiperparámetros y el despliegue de modelos probablemente preferirán usar SageMaker directamente u otras plataformas de ML.
- **Experimentación intensiva y ciclos de desarrollo rápidos:** Aunque Redshift ML facilita la creación inicial de modelos, los ciclos de experimentación y ajuste pueden ser más lentos en comparación con entornos de desarrollo de ML dedicados.
- **Implementaciones de inferencia en tiempo real con baja latencia:** Para aplicaciones que requieren predicciones instantáneas, desplegar el modelo como un endpoint en SageMaker o utilizar otros servicios de inferencia podría ser más adecuado.
- **Necesidad de explicabilidad y monitoreo avanzados:** Si bien Redshift ML ofrece cierta visibilidad a través de SageMaker, las capacidades avanzadas de explicabilidad de modelos (como SHAP o LIME) y las herramientas de monitoreo especializadas pueden no estar tan integradas o ser tan completas como en SageMaker.

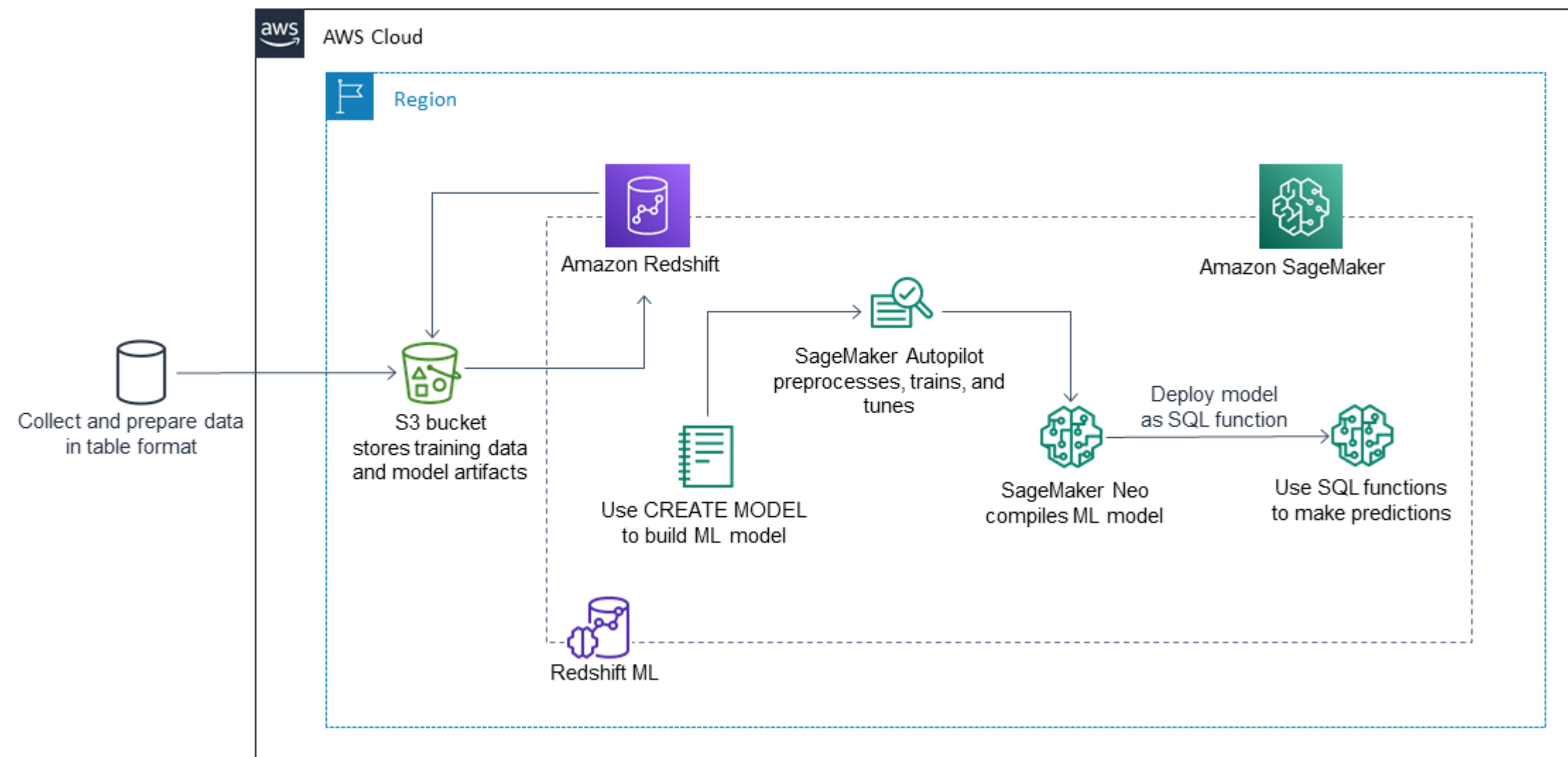
Ventajas para un Data Scientist:

- **Rapidez en la creación de modelos iniciales:** Permite construir prototipos rápidamente sin salir del entorno de datos.
- **Facilidad de acceso a los datos:** Elimina la necesidad de mover grandes volúmenes de datos para el entrenamiento.
- **Integración sencilla con el ecosistema de datos:** Facilita la incorporación de predicciones en los flujos de trabajo de datos existentes en Redshift.
- **Colaboración mejorada:** Permite a los analistas de datos consumir modelos de ML de forma más sencilla.
- **Menor curva de aprendizaje inicial:** Para tareas estándar, la interfaz SQL puede ser más accesible que las APIs de ML.

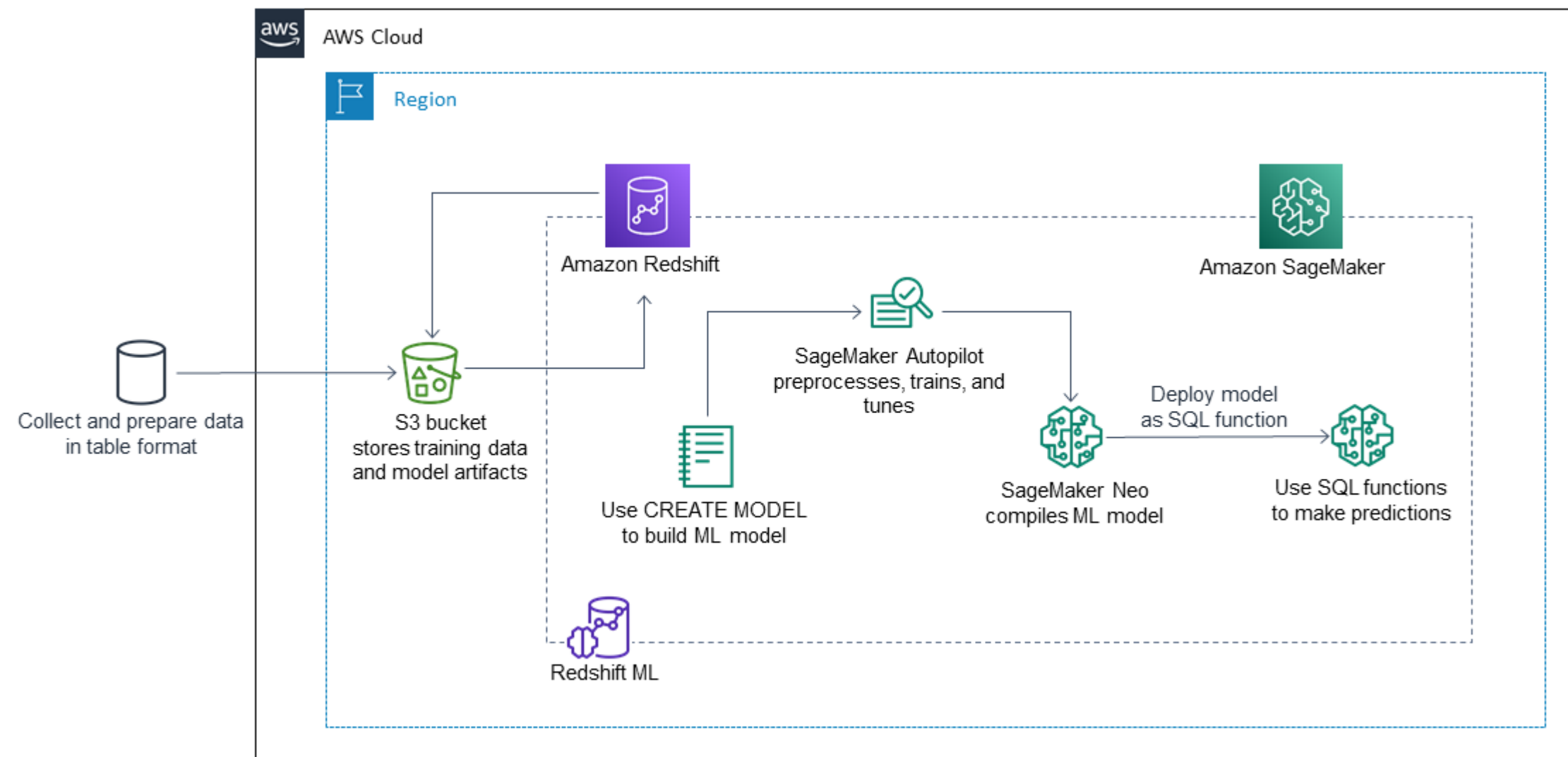
Inconvenientes para un Data Scientist:

- **Limitaciones en la elección de algoritmos y personalización:** La dependencia de SageMaker Autopilot puede restringir la experimentación con modelos avanzados.
- **Menor control sobre el proceso de ML:** Abstrae muchos detalles del entrenamiento y despliegue, lo que puede ser limitante para la investigación y el desarrollo.
- **Potencialmente menos eficiente para tareas complejas:** Para modelos sofisticados, usar SageMaker directamente puede ofrecer más flexibilidad y optimización.
- **Capacidades de explicabilidad y monitoreo limitadas:** Puede carecer de las herramientas avanzadas disponibles en plataformas de ML dedicadas.
- **Dependencia del ecosistema de AWS:** Está inherentemente ligado a los servicios de AWS.

Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.



Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.



El flujo de trabajo , es el siguiente:

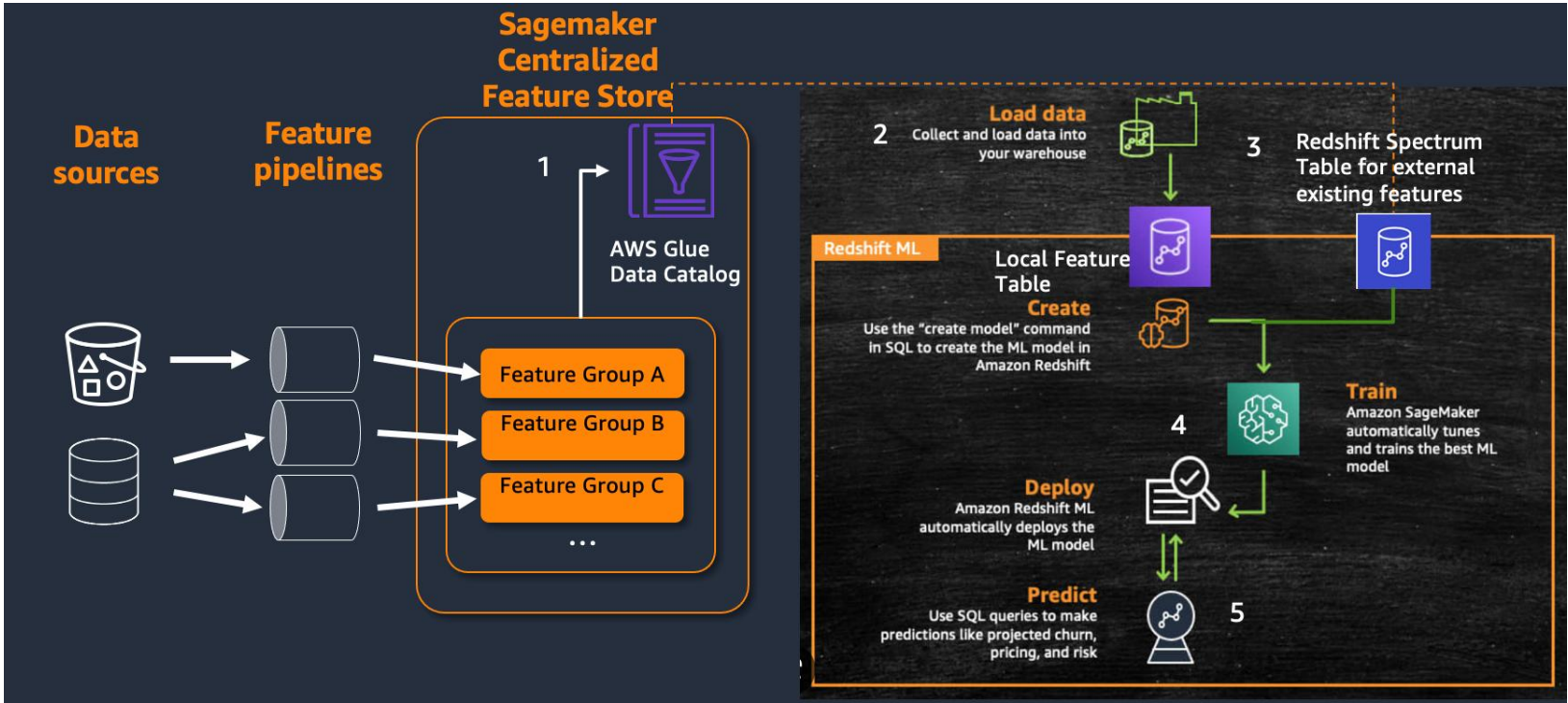
Amazon Redshift exporta los datos de ENTRENAMEINTO a Amazon S3.
Amazon SageMaker Autopilot procesa previamente los datos de ENTRENMAIENTO El *procesamiento* previo cumple funciones esenciales, tales como la imputación de los valores faltantes.

Reconoce que ciertas columnas son categóricas (como el código postal), les da el formato adecuado para la formación y realiza muchas otras tareas. Elegir los mejores preprocesadores que se aplicarán en el conjunto de datos de ENTRENAMEINTO supone un problema en sí mismo, por lo que Amazon SageMaker Autopilot automatiza su solución.

Amazon SageMaker Autopilot encuentra el algoritmo y los hiperparámetros del algoritmo que ofrecen el modelo con las predicciones más precisas.

Amazon Redshift registra la función de predicción como una función SQL en el clúster de Amazon Redshift.
Cuando se ejecutan instrucciones CREATE MODEL, Amazon Redshift utiliza Amazon SageMaker para EL ENTRENAMEINTO. Por lo tanto, la formación de su modelo conlleva un costo adicional.

También se paga por el almacenamiento utilizado en Amazon S3 para guardar los datos de formación. No se cobra la inferencia mediante modelos creados con CREATE MODEL que puede compilar y ejecutar en su clúster de Redshift. No se aplican cargos adicionales a Amazon Redshift por utilizar Amazon Redshift ML.



Amazon Redshift is columnar

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Presiona Esc para salir de pantalla completa

Terminology and Concepts: Columnar

Amazon Redshift uses a columnar architecture for storing data on disk

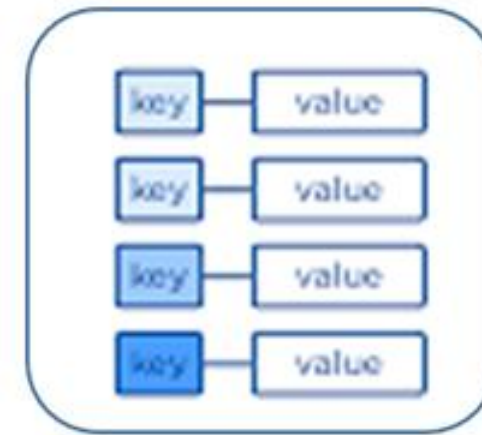
Goal: reduce I/O for analytics queries

Physically store data on disk by column rather than row

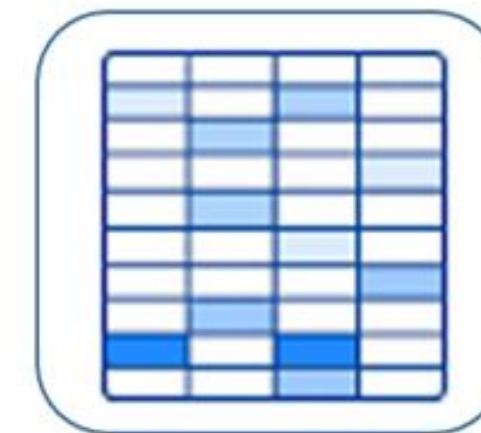
Only read the column data that is required



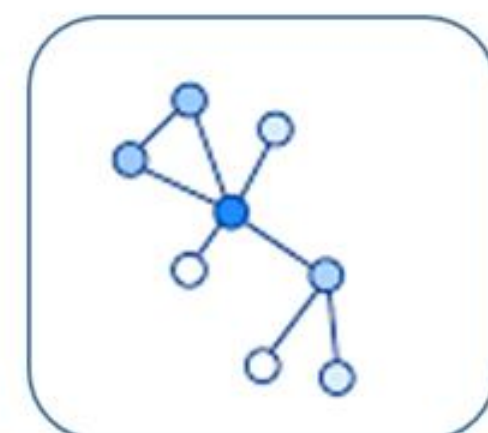
Document Store



Key-Value Store



Wide-Column Store



Graph Store

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Desliza hacia abajo para ver más detalles

aws



6:00 / 51:05 • Terminology and Concepts: Columnar



Amazon Redshift is columnar

Almacenamiento en columnas - Amazon Redshift

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (  
  aid INT      --audience_id  
  ,loc CHAR(3) --location  
  ,dt DATE     --date  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

aid

loc

dt

SELECT min(dt) FROM deep_dive;

Row-based storage

- Need to read everything
- Unnecessary I/O

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (  
  aid INT      --audience_id  
  ,loc CHAR(3) --location  
  ,dt DATE     --date  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

aid

loc

dt

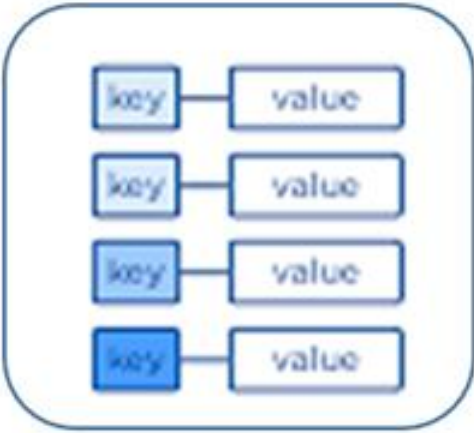
SELECT min(dt) FROM deep_dive;

Column-based storage

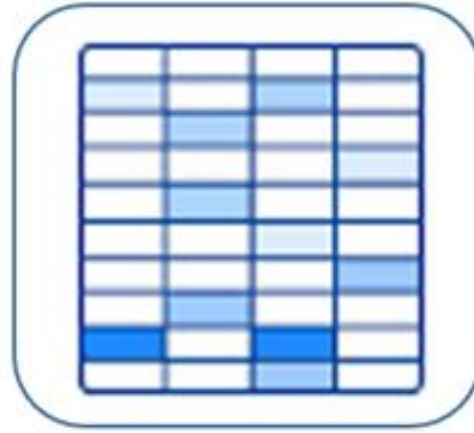
- Only scan blocks for relevant column



Document Store



Key-Value Store



Wide-Column Store



Graph Store

Amazon Redshift compression

Deep Dive on Amazon Redshift - AWS Online Tech Talks




Compression: Example

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
,loc CHAR(3)  --location
,dt  DATE     --date
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

aid	loc	dt

Add 1 of 11 different encodings to each column

Botón de reproducción (k)

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Desliza hacia abajo para ver más detalles

aws

9:00 / 51:05 • Compression: Example >

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Presiona **Esc** para salir de pantalla completa

Compression: Example

```
CREATE TABLE deep_dive (  
    aid INT          ENCODE ZSTD  
    ,loc CHAR(3)     ENCODE BYTEDICT  
    ,dt  DATE        ENCODE RUNLENGTH  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

aid

loc

dt

More efficient compression is due to storing the same data type in the columnar architecture

Columns grow and shrink independently

Reduces storage requirements

Reduces I/O

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

9:12 / 51:05 • Compression: Example >

aws

TIGHT LOCK

Amazon Redshift data sorting

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Data Sorting

Presiona **Esc** para salir de pantalla completa

Goal: make queries run faster by increasing the effectiveness of zone maps and reducing I/O

Impact: enables range-restricted scans to prune blocks by leveraging zone maps

Achieved with a SORTKEY defined on one or more columns

Optimal sort key is dependent on:

- Query patterns
- Business requirements
- Data profile

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Sort Key: Example

Presiona **Esc** para salir de pantalla completa

Create TABLE deep_dive (

```
aid INT      --audience_id
,loc CHAR(3) --location
,dt DATE     --date
) SORT KEY (dt, loc);
```

Add a sort key to one or more columns to physically sort the data on disk

deep_dive		
aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

deep_dive (sorted)		
aid	loc	dt
3	SFO	2017-04-01
4	JFK	2017-05-14
2	JFK	2017-10-20
1	SFO	2017-10-20

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

<https://youtu.be/Hur-p3kGDTA?t=720>

Amazon Redshift data distribution

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Data Distribution

Distribution style is a table property which dictates how that table's data is distributed throughout the cluster:

- **KEY:** value is hashed, same value goes to same location (slice)
- **ALL:** full table data goes to the first slice of every node
- **EVEN:** round robin

Goals:

- Distribute data evenly for parallel processing
- Minimize data movement during query processing

key and what we essentially do is we take

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. 17:51 / 51:05 • Best Practices: Sort Keys >

<https://youtu.be/Hur-p3kGDTA?t=1071>

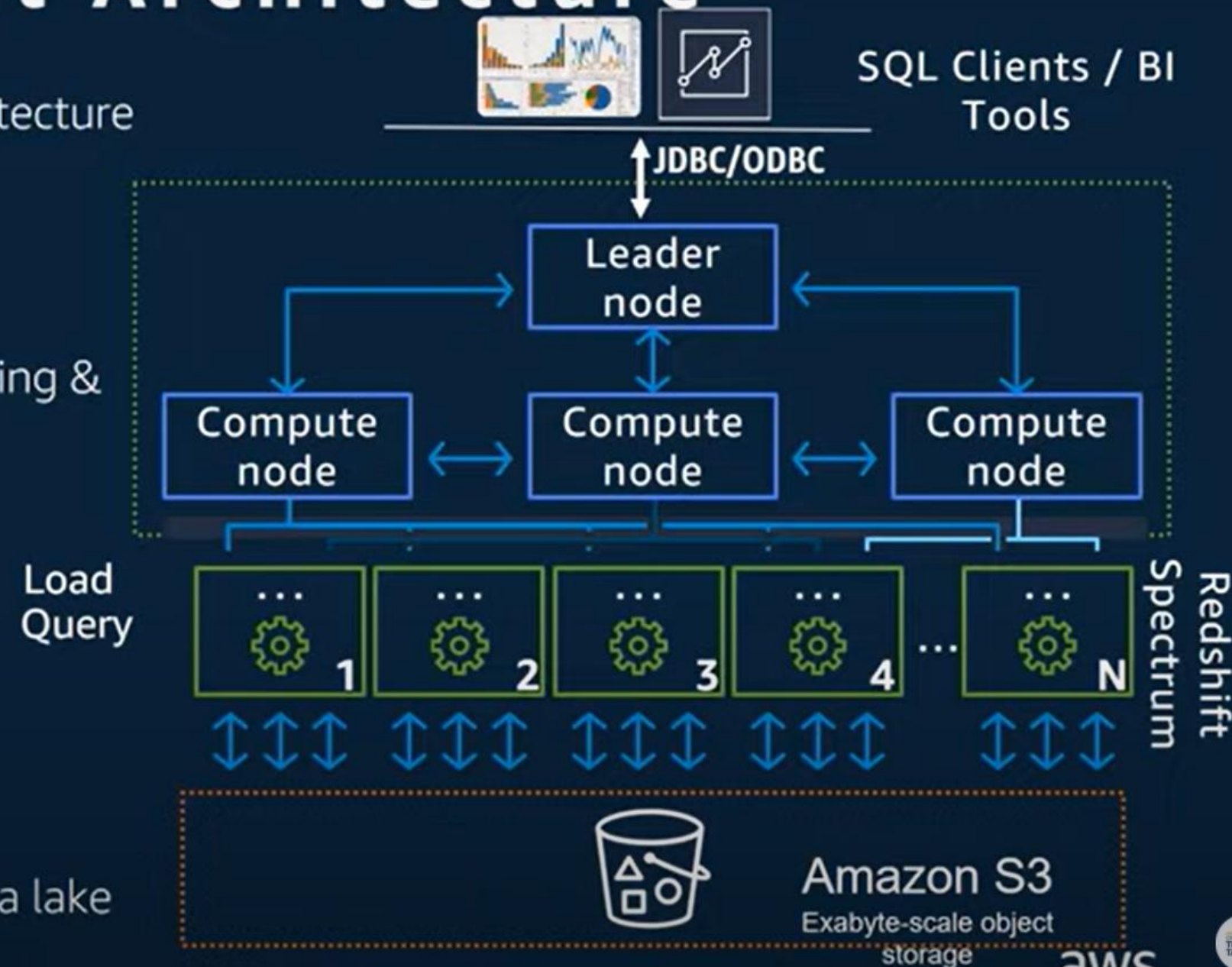
Amazon Redshift Architecture

Getting Started with Amazon Redshift - AWS Online Tech Talks

Amazon Redshift Architecture

Massively parallel, shared nothing architecture

- Leader node
 - SQL endpoint
 - Stores metadata
 - Coordinates parallel SQL processing & ML optimizations
- Compute nodes
 - Local, columnar storage
 - Executes queries in parallel
 - Load, unload, backup, restore from S3
- Amazon Redshift Spectrum nodes
Execute queries directly against data lake



Arquitectura del sistema de almacenamiento de datos - Amazon Redshift

https://youtu.be/_y0YHG95MI48?t=124

Minuto 2

Amazon Redshift slices

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Slices

Presiona **Esc** para salir de pantalla completa

A *slice* can be thought of like a virtual compute node

- Unit of data partitioning
- Parallel query processing

Facts about slices:

- Each compute node has either 2, 16, or 32 slices
- Table rows are distributed to slices
- A slice processes only its own data

aws

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

17:06 / 51:05 • Best Practices: Sort Keys > Desliza hacia abajo para ver más detalles

16:49

[Introducción a Amazon Redshift - Aprender BIG DATA desde cero](https://youtu.be/Hur-p3kGDTA?t=1009)
<https://youtu.be/Hur-p3kGDTA?t=1009> min
16:49

Amazon Redshift Blocks

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Blocks

Presiona **Esc** para salir de pantalla completa

Column data is persisted to 1 MB immutable blocks

Blocks are individually encoded with 1 of 11 encodings

A full block can contain millions of values

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Desliza hacia abajo para ver más detalles

aws

10:47 / 51:05 • Terminology and Concepts: Blocks

<https://youtu.be/Hur-p3kGDTA?t=652> minuto 11:23

[Amazon Redshift: 3 Comprehensive Aspects \(hevodata.com\)](https://hevodata.com)

Amazon Redshift Zone Maps

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Zone Maps

Goal: eliminates unnecessary I/O

In-memory block metadata

- Contains per-block min and max values
- All blocks automatically have zone maps
- Effectively prunes blocks which cannot contain data for a given query

<https://youtu.be/13ilj34nkQE>

Gracias